CS132 Coursework

Archie Harrodine
2127935

Contents

1 Question 1.

Consider an active-low 8-to-3 encoder.

1.1 a.

Provide the truth table for the encoder. You should use appropriate labels for all inputs and outputs.

1.1.1 Encoder Research

When first seeing this problem, my knowledge on encoders were rather limited. My exposure to them had been exclusively in the digital logic lectures, and even then my understanding was lacking. What I roughly understood was that it took 8 inputs, and returned 3 outputs, as seen by the diagram below.

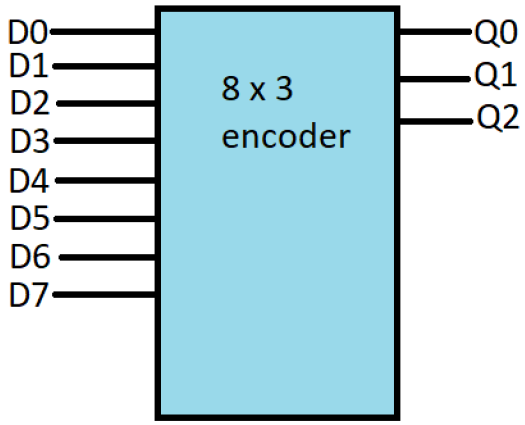


Figure 1: Image of encoder, inspired by images on Google.

However this on its own didn't help me that much, so I then proceeded to look into other examples of encoders, on the lectures. The one found was:

$Y0$	$Y1$	$Y2$	$Y3$	$X0$	$X1$
0	1	1	1	0	0
1	0	1	1	0	1
1	1	0	1	1	0
1	1	1	0	1	1

When looking at this, I was then unsure whether I was supposed to make it so exclusively those Y inputs gave those X outputs, or if all possible inputs of Y mapped to those 4 possible X outputs.

Taking to the internet at first confused me further, with the discussion of a priority encoder. However, eventually I managed to determine that what was wanted. An encoder is supposed to take 2^n possible inputs, and translate that into n outputs. This works because only one bit is selected at a time, which means that there will only be 2^n combinations, and not 256 (fortunately). Therefore for an 8-to-3 encoder, it was looking for an effective extension of the table of the 4-to-2 encoder, where in the event that only one value of Y is active, it would give a unique output. This is because only at one point will there ever be one value of Y active in the circuit, so we can ignore all other cases. What helped my understanding is the real world example of (something like) a keyboard, where only one key will be pressed at a time, and a character will be selected at that time.

1.1.2 Active Low Research

The only other area of uncertainty lay in what was meant by active low. From doing research and finding out that the table above is active low, I figured that it just implies that the inputs we usually associate with being active, actually is inactive. In terms of the table, it only meant that there would be a series of 1s, with only a single 0 (though it would have implications further on the circuitry). This made sense to me after I was exposed to potential real world use of reducing power usage, and perhaps making circuits simpler.

Finally, I also rewrote the inputs as I_N, and outputs as Y_N as that made more conceptual sense (I for inputs).

1.1.3 Truth Table

I_7	I_6	I_5	I_4	I_3	I_2	I_1	I_0	Y_2	Y_1	Y_0
1	1	1	1	1	1	1	0	0	0	0
1	1	1	1	1	1	0	1	0	0	1
1	1	1	1	1	0	1	1	0	1	0
1	1	1	1	0	1	1	1	0	1	1
1	1	1	0	1	1	1	1	1	0	0
1	1	0	1	1	1	1	1	1	0	1
1	0	1	1	1	1	1	1	1	1	0
0	1	1	1	1	1	1	1	1	1	1

As can be seen, the truth table of the encoder is complete, where my method for going through each permutation, was to imagine the Y output as binary, and to add 1 each time, providing unique outputs.

1.2 b.

Express the function of the encoder using Boolean expressions.

1.2.1 Initial Approaches

My first instinct upon seeing this question was to do a Karnaugh map. Then I realised how grid spaces there would be to fill out. Permutations of 4 variables, then squared, gives $(2^4)^2 = 256$ and that seemed highly inefficient.

So instead I looked at using Boolean logic, where I would define the series of positive outputs with a collection of and gates.

$Y_2 \equiv (I_0 I_1 I_2 ... \bar{I_7}) + (I_0 I_1 ... \bar{I_6} I_7) + (I_0 ... \bar{I_5} ... I_7) + (I_0 ... \bar{I_4} ... I_7)$
$\equiv I_0 I_1 I_2 I_3 (I_4 I_5 I_6 \bar{I_7} + I_4 I_5 \bar{I_6} I_7 + ...)$
$\equiv I_0 I_1 I_2 I_3 (I_5 I_6 (\bar{I_4} I_7 + I_4 \bar{I_7}) + I_4 I_7 (\bar{I_5} I_6 + I_5 \bar{I_6}))$
It was at this point in the proof I realised something very important. There had to be a better way.

1.2.2 Proof by Inspection

Studying the truth table, it hit me that I didn't need to do a bunch of difficult simplification - *that was for question 3* - rather I could just look closely at the table. Looking exclusively at Y_2, I can see that it is 1 when either I_7 or I_6 or I_5 or I_4 is false; and that sounds like the natural language equivalent of $Y_2 \equiv \bar{I_7} + \bar{I_6} + \bar{I_5} + \bar{I_4}$

By tracing the table where the outputs are 1, and following to which input is 'active' a series of or statements can connect all of the inputs, resulting in these three solutions for the circuits:

$Y_2 \equiv \bar{I_7} + \bar{I_6} + \bar{I_5} + \bar{I_4}$
$Y_1 \equiv \bar{I_7} + \bar{I_6} + \bar{I_3} + \bar{I_2}$
$Y_0 \equiv \bar{I_7} + \bar{I_5} + \bar{I_3} + \bar{I_1}$

1.3 c.

Design a logic circuit to implement the encoder using only NOR gates. You should explain how you arrived at your design and

how it can be tested.

1.3.1 Circuit Identities

For this question, it was a simple case of reworking the functions previously worked out, and condensing them into a series of NOR gates. For this I needed two identities:

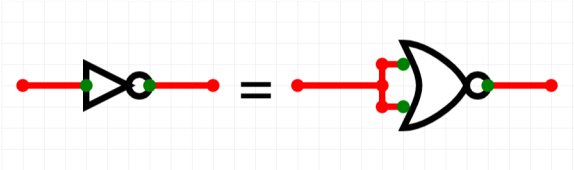


Figure 2: How to write a NOT gate as a NOR gate.

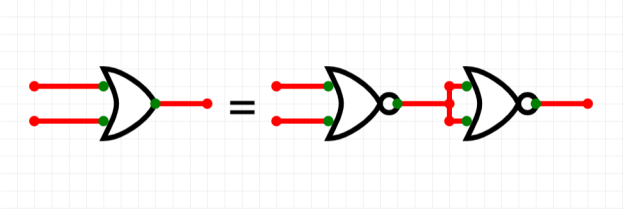


Figure 3: How to write an OR gate with NOR gates.

1.3.2 Basic Gates Logic Circuit

With these everything needed to assemble the correct circuit was here, as all my previous expression consisted of were a series of ORs and NOTs (Due to active low). I did pause to wonder whether active low needed to be accounted for, however I'm quite certain it needs to be (by applying a NOT gate at the start), as it isn't like the logic gates are any different than usual.

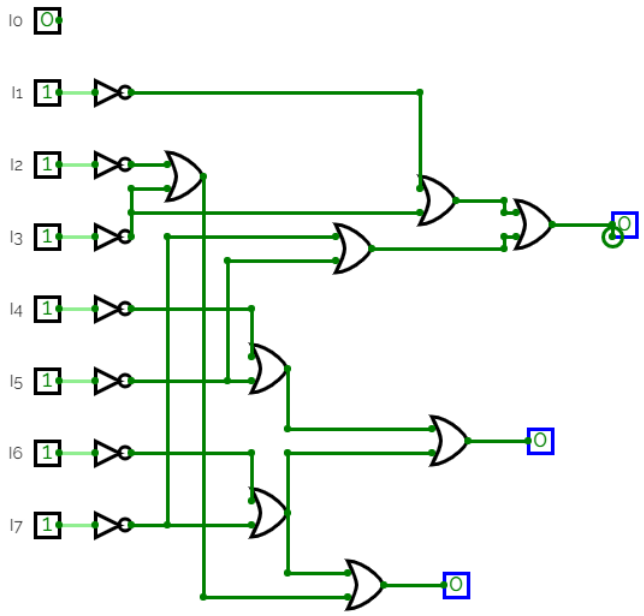


Figure 4: Logic circuit using basic gates.

1.3.3 NOR Gate Logic Circuit

This logic circuit shows how to write the circuit without NOR gates; so now using the identities we can rewrite the same circuit like this:

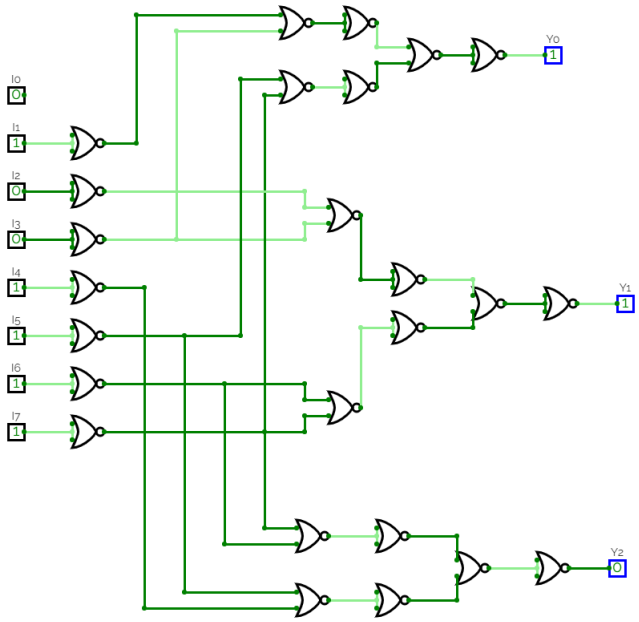


Figure 5: Logic circuit only using NOR gates.

This diagram, is incredibly disorganised and difficult to follow, additionally it isn't the most effective logic circuit as there are ways to minimize the number of NOR gates present. There are two approaches I can take this, either I can optimize the number of NOR gates used, or I can optimize the readability of the diagram.

1.3.4 Optimisation

If I were to focus on purely making this diagram more readable, I would condense the NOR gates into 4 input NOR gates. However there are a few issues in doing this. Firstly, finding an image for a 4 input NOR gate is incredibly difficult, and secondly it should not be the focus. When physically designing circuits, it is important to optimize the cost, and therefore by reducing the number of gates used, it reduces the amount of cost required to encode a Boolean expression.

Therefore, there are a few areas of the diagram that can be improved upon. Firstly, I looked for any matching terms between Y_0, Y_1 and Y_2. This is because currently, the terms are taken all in isolation, which means there may be repeated gates. This avails itself with $\bar{I_7} \vee \bar{I_6}$ appearing in both Y_2 and Y_1. Alternatively I could have matched $\bar{I_7} \vee \bar{I_5}$ shared by Y_2 and Y_0.

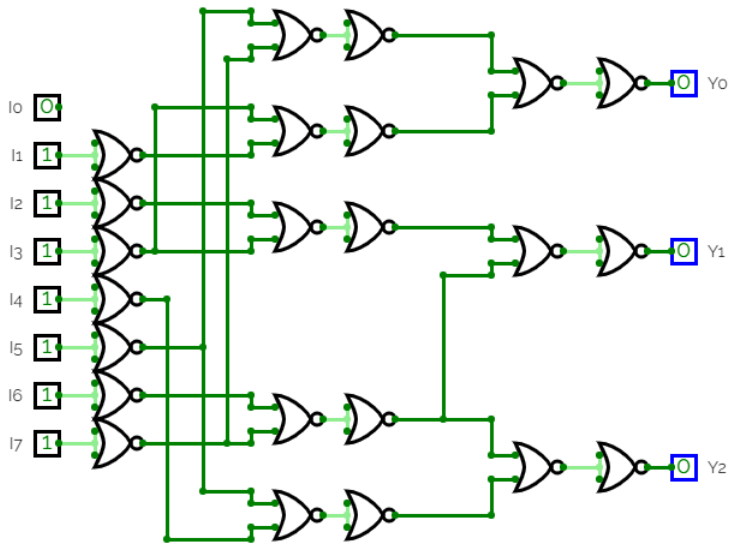


Figure 6: Refined logic circuit using only NOR gates.

By doing some small modifications it definitely improved the look and the amount of gates that the circuit has, however now without fundamentally what the circuit does, there is little to optimize it further.

That is if I do not consider what the function of a 8 to 3 encoder is. An 8 to 3 encoder's main role is to give unique output to different inputs. One thing that doesn't contribute to the uniqueness is the the NOR gate on the end. At the end of each branch of the circuit, just before each output, is the equivalence of a NOT gate, which simply toggles the value. As all 3 are toggled, if all 3 were toggled again, then all answers would still be unique. And by law of double negation, this allows for the removal of those final NOR gates. This is at a slight catch though. The question requests for a circuit that implements the original encoder designed, which the circuit above achieves. A toggled version of this encoder can then be made with the circuit below:

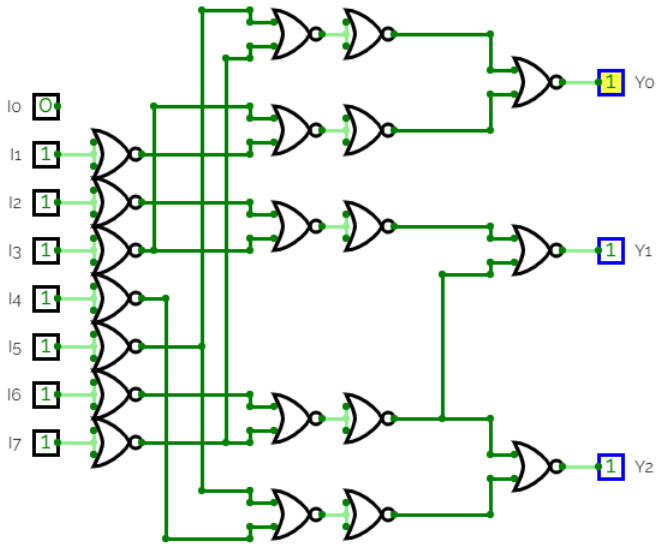


Figure 7: Refined toggled logic circuit using only NOR gates.

This however does result in this truth table:

I_7	I_6	I_5	I_4	I_3	I_2	I_1	I_0	Y_2	Y_1	Y_0
1	1	1	1	1	1	1	0	1	1	1
1	1	1	1	1	1	0	1	1	1	0
1	1	1	1	1	0	1	1	1	0	1
1	1	1	1	0	1	1	1	1	0	0
1	1	1	0	1	1	1	1	0	1	1
1	1	0	1	1	1	1	1	0	1	0
1	0	1	1	1	1	1	1	0	0	1
0	1	1	1	1	1	1	1	0	0	0

Reducing the number of total NOR gates to 20. This certainly isn't the most reduced it could be, as I am sure there are other improvements that could be made, especially regarding the first section, which is comprised of all 'NOT' gates due to the active low state of the circuit.

1.3.5 Testing

Finally, all that was left to do was to test it, to ensure that it actually gave the correct values. To specify, I tested the original circuit with the NOT gates at the end, which meant for $I_0 = 0, Y_0 = Y_1 = Y_2 = 0$, and

so on (see original truth table). At first, I went through each and every interaction and gate, labelling the inputs, and what was spat out of each gate. This availed the same correct truth table, yet I wasn't satisfied, since I am very prone to making mistakes. Therefore, I made the circuit in logicly.ly (a website used for logic diagrams), in which I considered then using the truth table function however, with 8 inputs, there are 256 possible combinations, in which only 8 we care about. It is far easier to flick the switches and show in the diagram below:

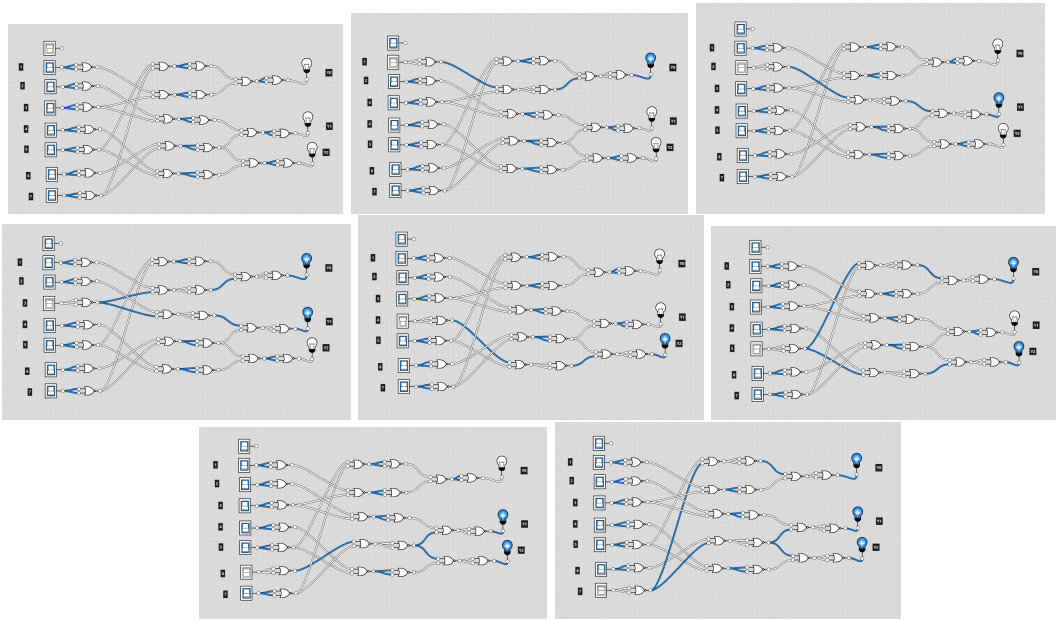


Figure 8: Test cases using logicly.ly.

These diagrams have a lightbulb at the end to represent if it is a one (on) or a zero (off). This follows the original truth table exactly, showing the original diagram's success. Far easier than writing out each value by hand.

2 Question 2.

We have studied shift registers in the context of sequential logic circuits. A bidirectional shift register is one in which data can be shifted left or right on the basis of a control input.

2.1 a.

Design a 4-bit bidirectional shift register using the logic components studied in CS132. Your design should be serial-in serial-out. You should use appropriate labels for all inputs and outputs.

2.1.1 Sequential Logic Research

Similar to the encoder, I had no idea what I was doing at first. Very quickly the problem diverged into a research project, in which I would have to use the vast knowledge of the internet to guide my understanding. The only problem, is that I didn't even understand a regular register.

When covering sequential logic, my understanding was entirely lacking and limited, with little understanding as to how values were stored or updated or changed, let alone how to shift values in a certain direction. Taking to the original lectures I looked into wat an n-bit register looked like, and there I began to gain an understanding of how to tackle the question.

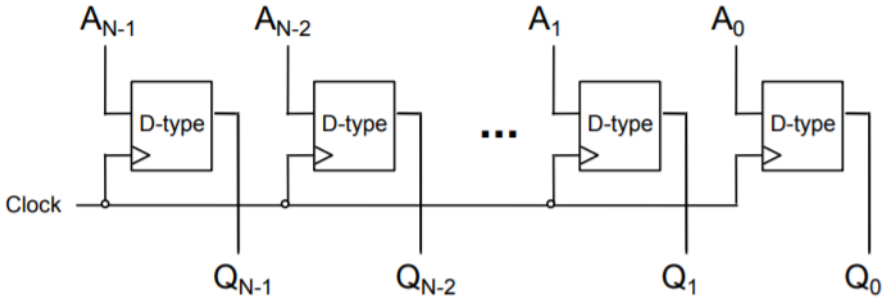


Figure 9: Image of N-bit register from Digital Logic Lecture.

Above is an image of the n-bit register, which takes in multiple inputs, and spits multiple outputs. What is important to the design of the register is a few key components. Firstly the D type flip flop box takes in the two inputs, and spits out two inputs, however for our purposes, one of those outputs can be ignored. A clock must connect to each flip flop, and it must take the new input of a changing binary value (Those are

the A values). The Q values meanwhile are the outputs from what was in the D type flip flop previously. With this basic understanding I could now begin to form an image of how this was to work. If there were to be a left shift, I was going to need the output of the first flip flop to go into the input of the next flip flop.

2.1.2 Serial vs Parallel

Before I could go forward though, there was one area that needed a great improvement in understanding, and that was what was meant by serial in, serial out. Doing some research and looking at examples of serial in serial out n-bit registers, I saw that it was single input, single output. The website electrical4u demonstrated this best to me, using this image to show a serial in, serial out n-bit register.

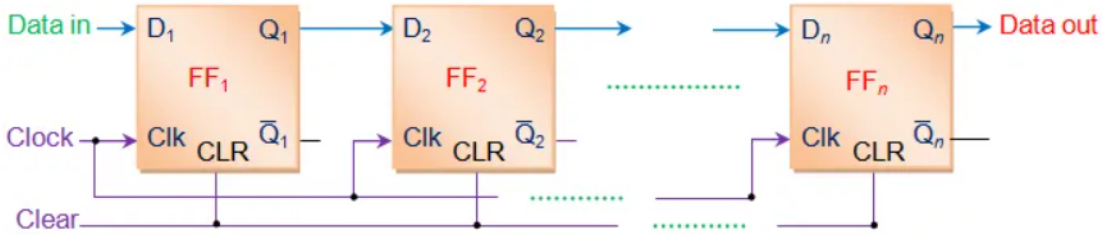


Figure 10: Serial in, serial out register from electrical4u.

As we can see there is one input into the data, and one input of the data out. The way it is supposed to work is if you have an n bit input, then you enter the least significant bit first, then repeatedly enter the next bit on each clock cycle, each time the previous LSB being outputted. The one part of the diagram I am choosing to ignore is the clear input, as whilst its function is fairly clear (pun was not intentional), it did not seem to have a place in my circuitry, in what we had been taught so far. When looking at what this circuit does, my gut instinct was to say this already filled the criteria. As when data was inputted, it would shift all bits to the right with a serial load, and output the bit lost. However, this was only half of the solution, as somehow this needed to go two ways. This is where the bidirectional part of the requirement came in, and so my research resumed.

2.1.3 Bidirectional Shift Research

The website javapoint, and several YouTube videos that gave worked through examples aided in helping to understand how the shift register should work. Since it needs to work in two directions, first a way to specify which way is required. This is quite simply solved by implementing a 'mode' which is just one more input to deal with. Afterwards, it needs to take in two inputs, both a right shift input or a left shift input. Then we need to get a left shift output and a right shift output from the 'end' registers. Finally, we need some operation to decide which way, which would include the mode, the input coming from the right, and input coming from the left (which may be outputs of other registers). This diagram below demonstrates what is wanted, where the black box is the unknown operation of what we want.

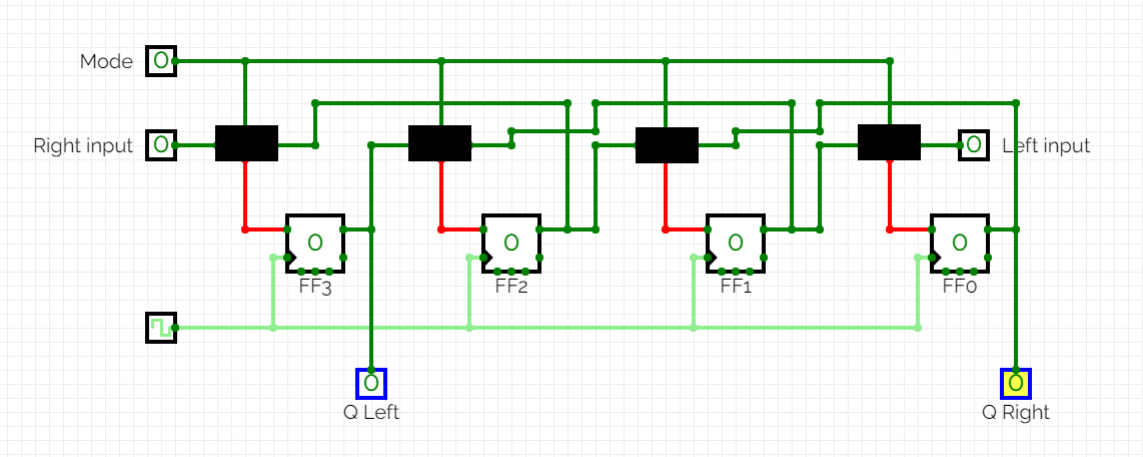


Figure 11: Bidirectional shift register, with unknown operations represented by the black box.

There were two ways I could go about this. I could either copy what was on the internet blindly, or I could make it make sense. If we assume that when mode (m) = 1, that it is a right shift, and when m = 0 it is a left shift, we can actually produce a truth table for what we want the output of the black box to be. Note: take right entry = R, and left entry = L, and the output = Q.

m	Q
0	L
1	R

m	R	L	Q
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

From here we can use the Boolean algebra we know and love:
$Q = (\bar{m} \wedge \bar{R} \wedge L) \vee (\bar{m} \wedge R \wedge L) \vee (m \wedge R \wedge \bar{L}) \vee (m \wedge R \wedge L)$
Using the commutative law we can rearrange this into these two collections:
$= (m \wedge R \wedge \bar{L}) \vee (m \wedge R \wedge L) \vee (\bar{m} \wedge \bar{R} \wedge L) \vee (\bar{m} \wedge R \wedge L)$
Then by the distributive law we can say:
$= (m \wedge R) \wedge (\bar{L} \vee L) \vee (\bar{m} \wedge L) \wedge (\bar{R} \vee R)$
Using the complement law and the identity law we simplify the statement down further to:
$= (m \wedge R) \vee (\bar{m} \wedge L)$
$= (m \wedge R) \vee (\bar{m} \wedge L)$

2.1.4 Testing the Design

And fortunately, this is the circuit design that was found online. For an example of how this would work in the circuit, if we look back to the first diagram we see the black box furthest on the right taking in the mode, left shift input, and the output from FF1, which we shall call Q_1. We shall look at both cases of the mode.
Case 1: $m = 0$
$(0 \wedge Q_1) \vee (1 \wedge L)$
$= 0 \vee L$
$= L$
And so the left input will enter the flip flop, meaning a left shift has occurred.
Case 2: $m = 1$
$(1 \wedge Q_1) \vee (0 \wedge L)$
$= Q_1 \vee 0$

$= Q_1$

And so the right input from the previous register will enter the flip flop, meaning a right shift has occurred, and that left input is irrelevant.

2.1.5 Logic Diagram of the Shift Register

Now all that was left to do was to put it into circuit diagram form.

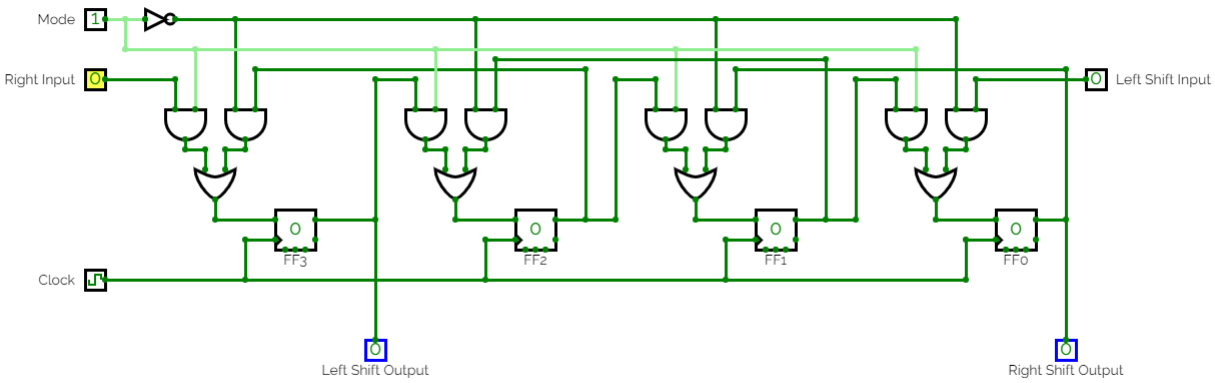


Figure 12: Bidirectional shift register, with correct operations.

Whilst the design for this was quite well done, there are a couple areas which didn't sit entirely right. The definition of serial in and serial out suggests that it has one input and one output. However, when looking at this we can see that there are two inputs and two outputs. This can be justified and explained away by the fact that one output and input is always redundant, but that in some ways is even less satisfying. Most internet sources do not go into this, and all reading I have done proposes that the design above is the correct one; that the two outputs and inputs are a result of the fact it is a bidirectional register, and so that is my final design. Especially when considering part usage and electronic design, it makes sense for there to be two inputs and outputs, with one being redundant, for example we see it in the program with the flip flops, as the inverses just are not used. Before continuing on to part b though, I do have a logical way to reduce the number of inputs and outputs to one:

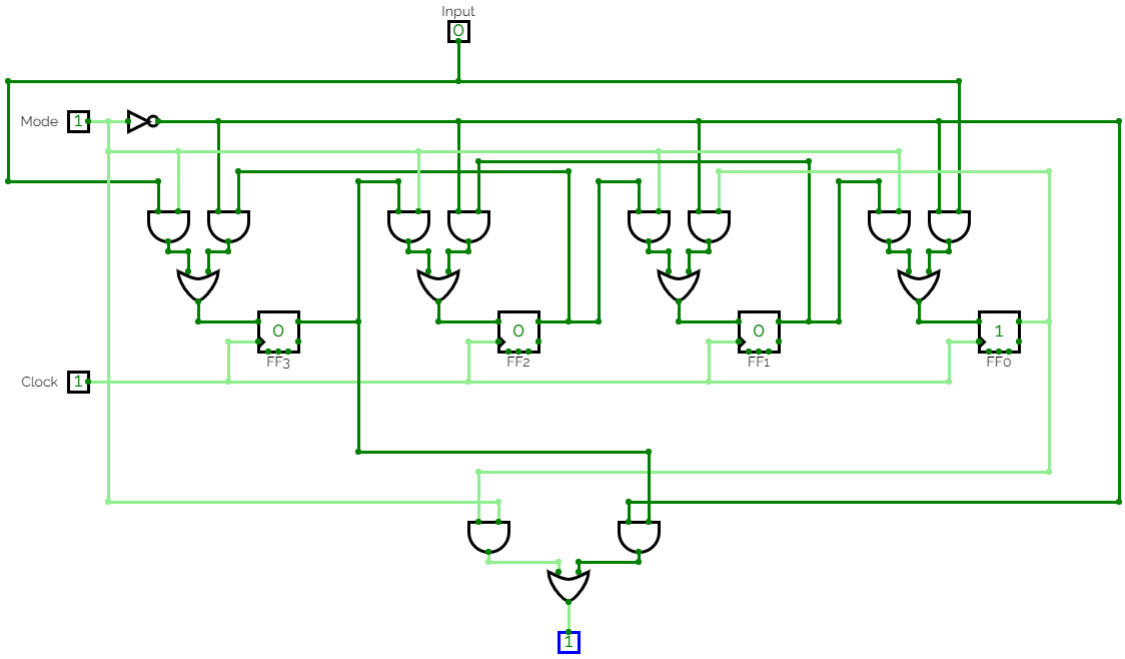


Figure 13: Bidirectional shift register, with modified input and output to allow for single input, single output.

The input section is fairly simply. When you take the input for a left or right shift, the value of the opposite input is irrelevant, as it is fazed out of existence at the AND gate. This means that by having both inputs be the same, it will not affect the outcome, only remove a redundant input. The output is more complicated. When there is a right shift, we want the output of the rightmost flip flop, and vice versa. So, we need to implement the opposite of what was done for the flip flops by doing these calculations: (Note value of FF3 is Q3, and value of FF0 is Q0)
$(\bar{m} \wedge Q3) \vee (m \wedge Q0)$
To demonstrate, suppose the mode is 1, we want to output Q0 as it is a right shift.
$(0 \wedge Q3) \vee (1 \wedge Q0)$
$= 0 \vee Q0$
$= Q0$
This works for Q3 as well then by definition.
These amendments though, as stated before, are likely a greater hindrance on the circuit as a whole, with even more gates to implement into the wiring. Hence, will not be included in the final design.

2.2 b.

Write an explanation of your design, detailing how it allows data to be shifted in each direction and discussing any assumptions it makes. You should use example inputs to illustrate circuit operation.

2.2.1 Pseudo-logic of the Shift Register

The previous part did explain the basics of how the circuit actually shifts data across, and inserts a new value of data in its stead. However it did not thoroughly detail the steps in which the bidirectional shift register actually works.

To write in a general form, we can say that it follows this process:
1. The mode is selected. If mode = 0 then it is to shift to the left. If mode = 1 then it is to shift to the right.
2. The bit is inputted in the left and in the right.
3. Starting at FF3, the current value stored at FF2 is taken, and entered into the function. Here this logical operation takes place:
(mode AND right input) OR (NOT mode AND current FF2).
4. The bit returned from this is stored in FF3 on the next rising clock edge.
5. This repeats for all 4 flip flops as it is a 4 bit register.

2.2.2 Visual Understanding of the Shift Register

This abstracted away form, is not very helpful. It doesn't explain how the values are distinguished, nor does it indicate how it all works. This will be best demonstrated by using a visual guide. The first case to go through is the general case to shift right. This will demonstrate how regardless of the value of the flip flop, they will shift over.

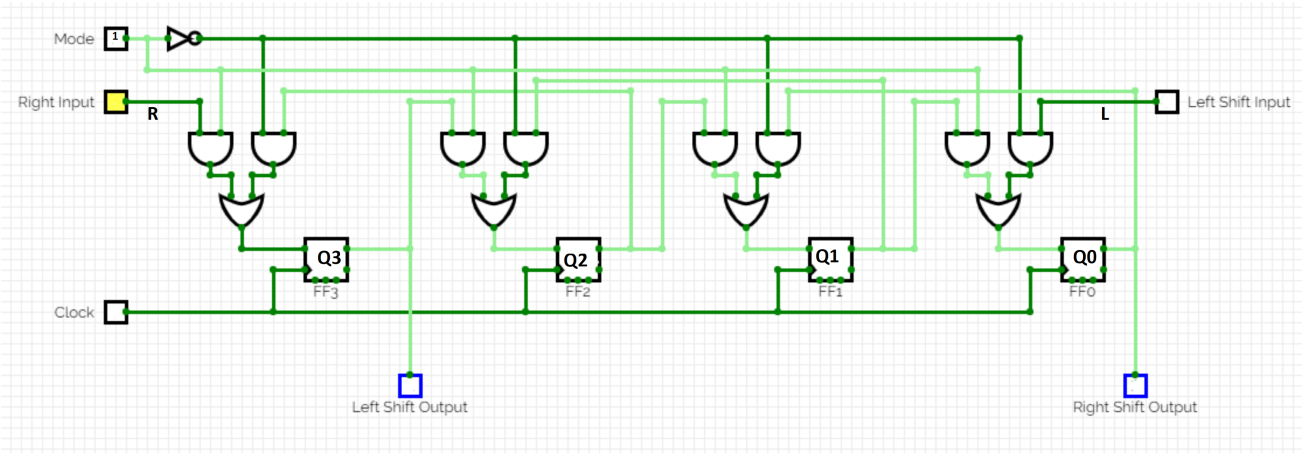


Figure 14: Step 1.

Step 1: This first diagram just showcases the initial starting values of the shift register. We have R and L defining the right and left inputs respectfully (where R and L are single bits). Then we have the mode set to 1, so this will be a right shift. Finally we have the registers set to Q3, Q2, Q1, and Q0, where these are all single bits.

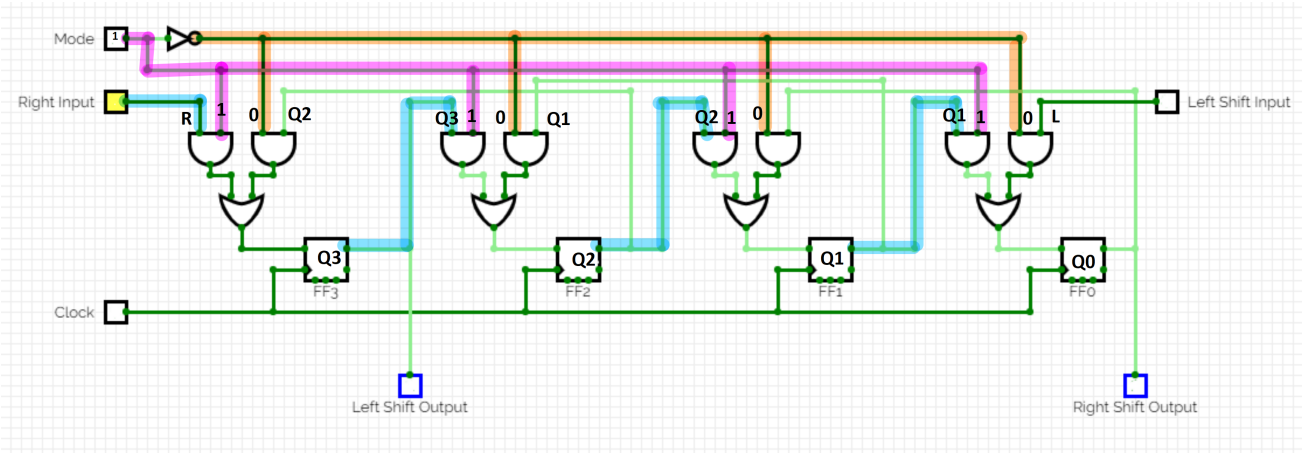


Figure 15: Step 2.

Step 2: Now that we have all the values defined we can follow where they go. Here we have the value of the mode (1) represented in pink, and the inverse of that value (0) represented in orange. The blue lines follow the other inputs of the right shift: R, Q3, Q2, and Q1. Originally, we had everything highlighted, but it was a convoluted mess. This puts focus on what values are going where. As we can see on the far left, we have R and 1 entering an AND, and 0 and Q2 entering an AND. This similarly

applies to the other 3 components.

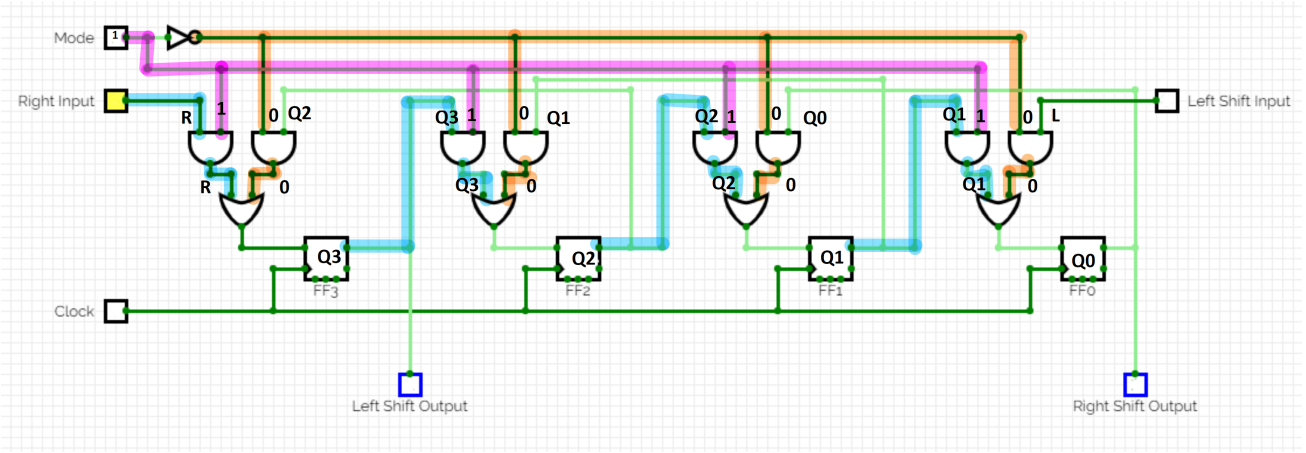


Figure 16: Step 3.

Step 3: This takes the next logical step and applies the bit-wise operations on the inputs, using some known identities. For example on the far left:
$R \wedge 1 = R$ always due to the identity law.
Similarly $0 \wedge Q2 = 0$ always due to the annulment law.
Here we can see all of the bits that were proposed to be shifted left, have all be negated out of existence, thereby leaving the only bits we care about. This follows for all 4 sections of the shift register.

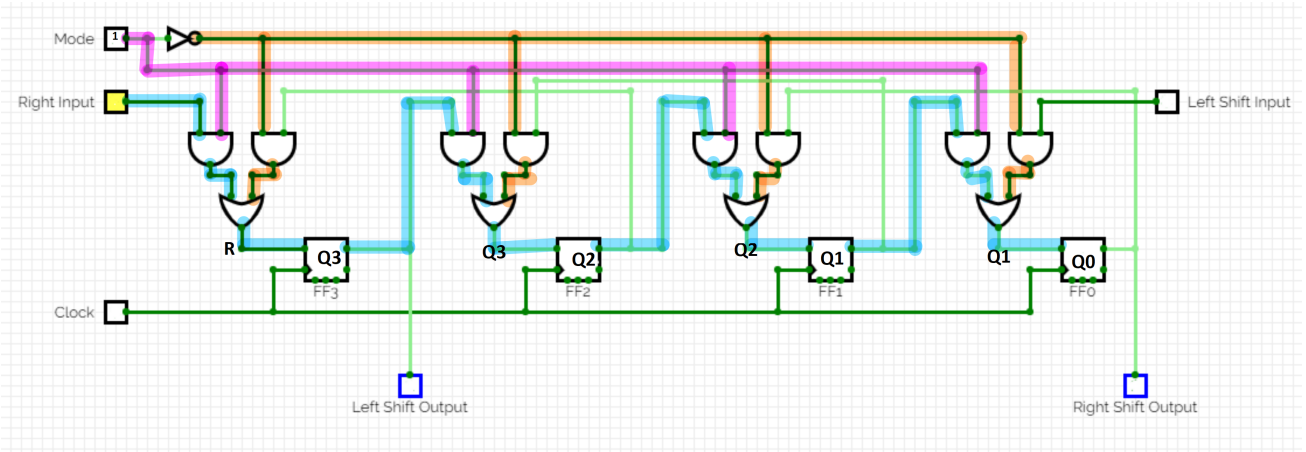


Figure 17: Step 4.

Step 4: This performs the next Boolean logic step off $R \vee 0$, $Q3 \vee 0$, etc. Due to the identity law again, we know that $A \vee 0 = A$ and hence the

output here will be the original bit entered. This way if it is a 0 or a 1, it will be accepted. This now has made its way to the actual flip flop to be entered.

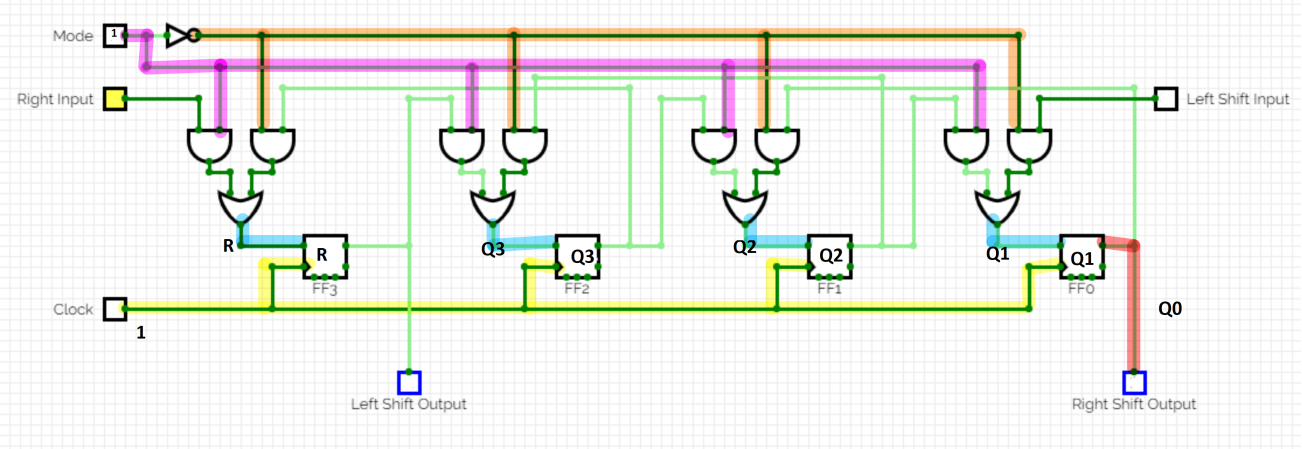


Figure 18: Step 5.

Step 5: This final step has the clock finally get involved. As the flip flop will only change its value on the next rising edge of the clock, once the clock sends its pulse it means that R, Q3, Q2 and Q1 are all stored in their intended locations. Regardless of bit, the location has changed appropriately. Additionally, the value of Q0 is flushed out to the right shift output, as it should be in a serial out register. This concludes the process, and it shows it holds for all values of the flip flops and the right input.

2.2.3 Left Shift with Example

I could demonstrate the same thing for the left input and despite my love of highlighting, there is no need to. The way that the left shift is designed is to be an identical version of the right shift, but going the other way. Both directions allow for the shift. To showcase the left shift, it can be done on the online circuit designer:

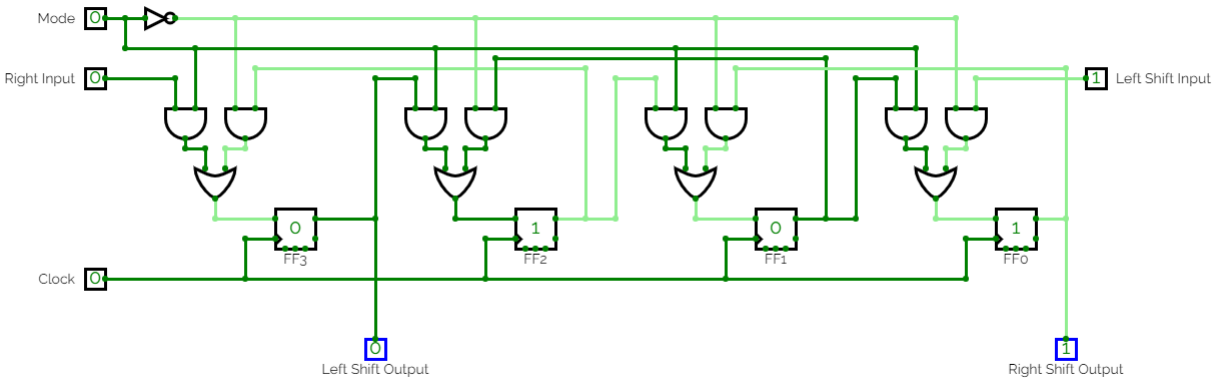


Figure 19: Demonstrating a left shift 1.

Here we see that currently 0101 is stored in the register. The mode is set to 0, so it will perform a left shift. The value of the left data is 1, and currently the clock value is 0. Upon the clock changing, then the shift will occur.

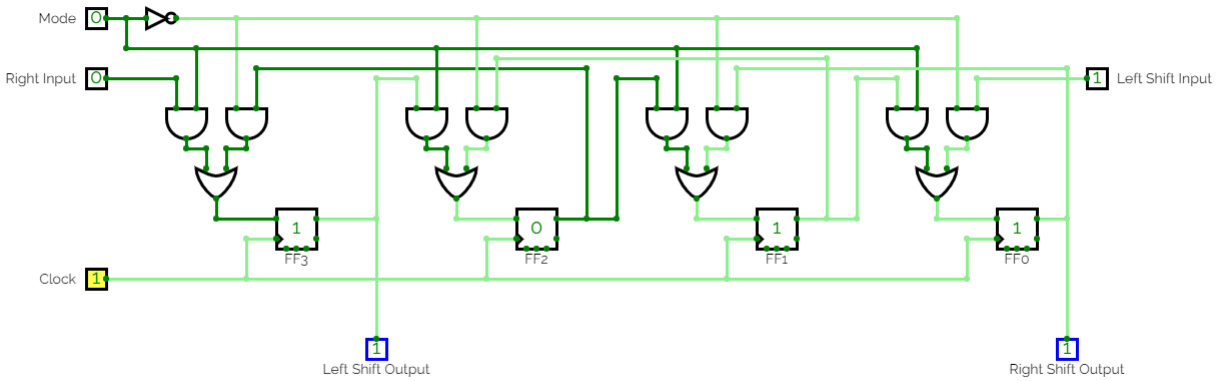


Figure 20: Demonstrating a left shift 2.

As can be seen, the bits have shifted to the left, with FF0 being the value inputted by the left input. This shows the shift as a success.

2.2.4 Assumptions Made

The final thing to address is the various assumptions that were made to design this register. The largest assumption was that it was operating in active high. This was assumed so that I would have an easier time

designing the logic gates, and so that I did not need to play around with active low and clocks. Another assumption made is that the inverses of the D type flip flop outputs, are not used outside of the flip flop. This reflects in the circuit diagram in which only one output is depicted.

2.3 c.

c. Outline how your design could be adapted to form an N-bit bidirectional shift register that is parallel-in parallel-out.

2.3.1 Research

Now that I had a better understanding of how registers work, and what is actually meant by serial and parallel, I was feeling more confident in how the N-bit directional shift register that is PIPO (Parallel in, parallel out). When considering an N-bit PIPO register, we see that each flip flop takes in a single input, and return out a single output.

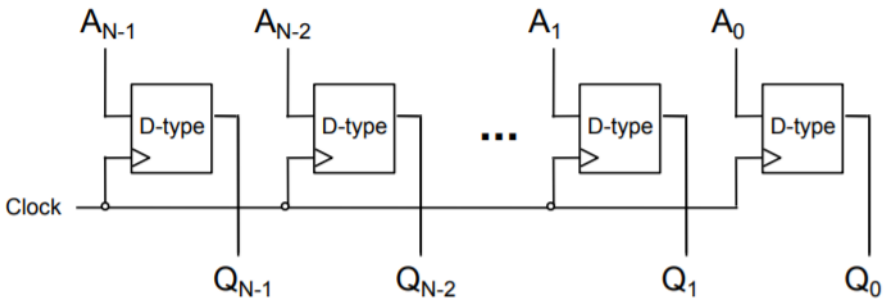


Figure 21: Parallel in, parallel out register.

The example demonstrated at the start showcases this, with there being N inputs and N outputs for this N bit register. But, how does this translate over to a bidirectional shift register. If N new inputs are entered at once, does that even cause a shift? Is the diagram above actually just a bidirectional shift register anyway? If we were to look at a SIPO (Serial in, parallel out) shift register shown similarly in the lecture:

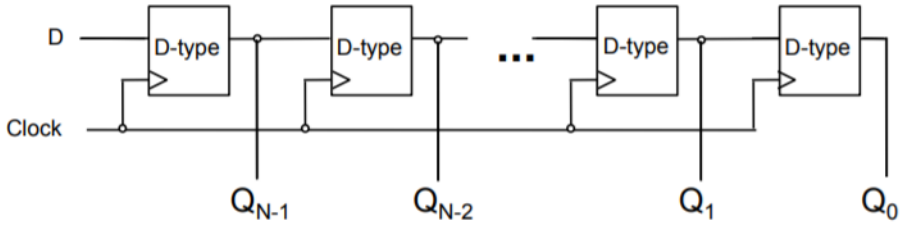


Figure 22: Serial in, parallel out shift register.

We see that the idea of how it is outputted makes sense to be a shift register, and if this was to be extended to be a parallel input, then everything would shift, just all at once.

2.3.2 Breaking it down

Although this made conceptual sense, there were parts that still bugged me. Firstly there was the bidirectional part, which doesn't make much sense in this context. Secondly, there is no 'shift' taking place. I suppose the values are being shifted out, but that isn't a satisfying conclusion. Hence, I began to research again.

The website electrical4u proposed one solution, discussing right shift and left shift shift registers using parallel inputs and outputs.

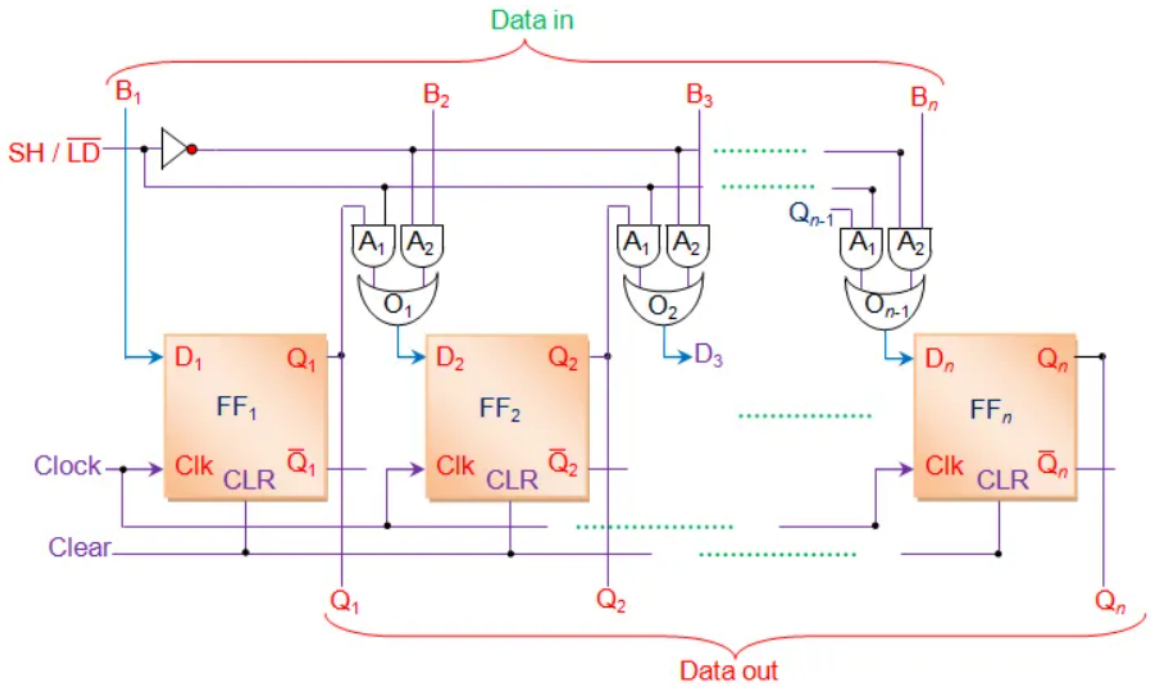


Figure 23: Right shift parallel load register from electric4u.

Above is the diagram shown on their website. This diagram shows a very similar one to the one implemented before, however it serves a very different purpose. When the SH/LD line is 1, this follows so that a shift occurs. This is because if we look at the logic gates once more, we get $(Q1 \wedge 1) \vee (0 \wedge B2) = Q1$ which tells us that a right shift is happening. When the SH/LD line is 0, that means B2, B3, all the way to Bn is loaded into the registers straight. This means that this register design has in theory both shift capabilities, whilst having the parallel load. So, now I need some way to implement this with both left and right shifts.

2.3.3 Creating a Design

Thinking about the design further, it would make sense then for there to be three modes: parallel load, right shift, and left shift. To achieve this we will need two 'mode' inputs, which means we will have to account for 4 cases:

m1	m2	Operation
0	0	?
0	1	Right shift
1	0	Left shift
1	1	Parallel Load

At first I was unsure at what should actually happen when both m1 and m2 are 0, but there is a simple solution, to do nothing.

Now with this framework in place, we need to do the output game once more. If we refer back to our previous strategy, we need to effectively create a sub-circuit that takes in the inputs of the two modes, the parallel load, the shift right load, and the shift left load. However, we also need one other load, and that is what is in the current register, as if we have a 0,0 mode case, then we want to feed that back in. Drawing this out, we have a truth table that is 7 x 64. Which, is a lot. By looking at what is at the end, we would end up with a Boolean expression of 32 terms. Initially, I drew out the table, and began noticing patterns, and how things could simplify down. But, in a moment of pure utter coincidence, when I was looking back through the digital logic lecture for a Boolean algebra law, I stumbled across multiplexers.

2.3.4 Multiplexers

In designing the large table of logic for how to simplify down the expression, I first wrote a simpler table detailing this:

m1	m2	Q
0	0	C
0	1	R
1	0	L
1	1	P

This should look familiar, as it is just rewriting the table up top but using letters to represent the action. Now in the lecture, there was a slide detailing multiplexers and their function, and it just so happened to show this table:

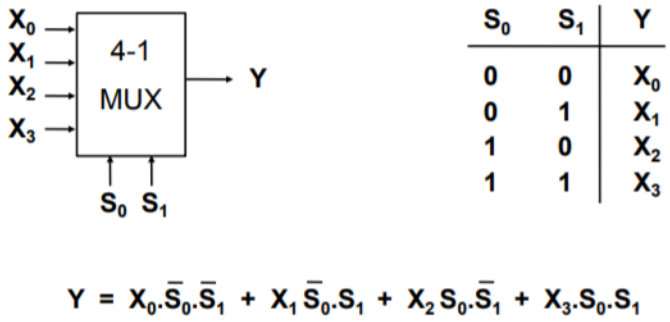


Figure 24: Snippet from Digital logic lecture detailing multiplexers.

And which to be fair to myself, I was on route to getting that Boolean expression, it just may have taken a bit longer. Regardless, now I had a foothold into how this can all work. By simply entering the 4 values into the multiplexer, and the two modes, the output is what would enter the register. So, now with the expression I can write what the output going into the register should be:
$Q = (C \wedge \overline{m1} \wedge \overline{m2}) \vee (R \wedge \overline{m1} \wedge m2) \vee (L \wedge m1 \wedge \overline{m2}) \vee (P \wedge m1 \wedge m2)$
Where we will use this to express the series of operations going on, so the diagram is not flooded.

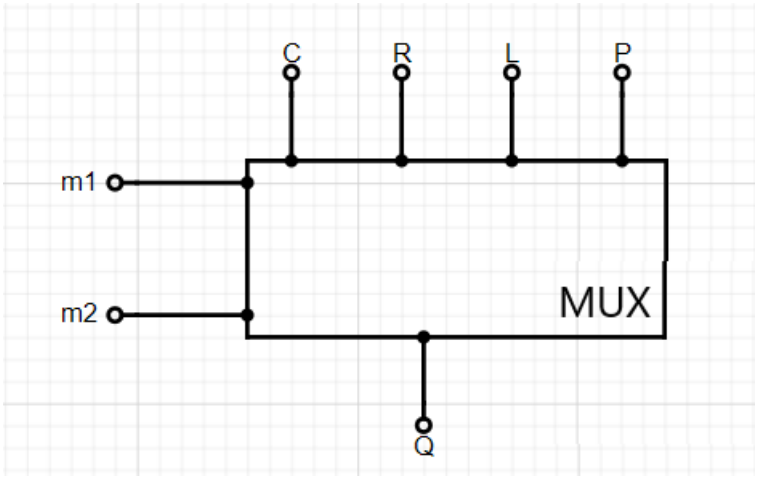


Figure 25: How multiplexers will be detailed in the register diagram.

2.3.5 Drawing the Logic Diagram

So finally to compile it all together, we get this circuit diagram for an N-bit bidirectional, parallel-in, parallel-out shift register.

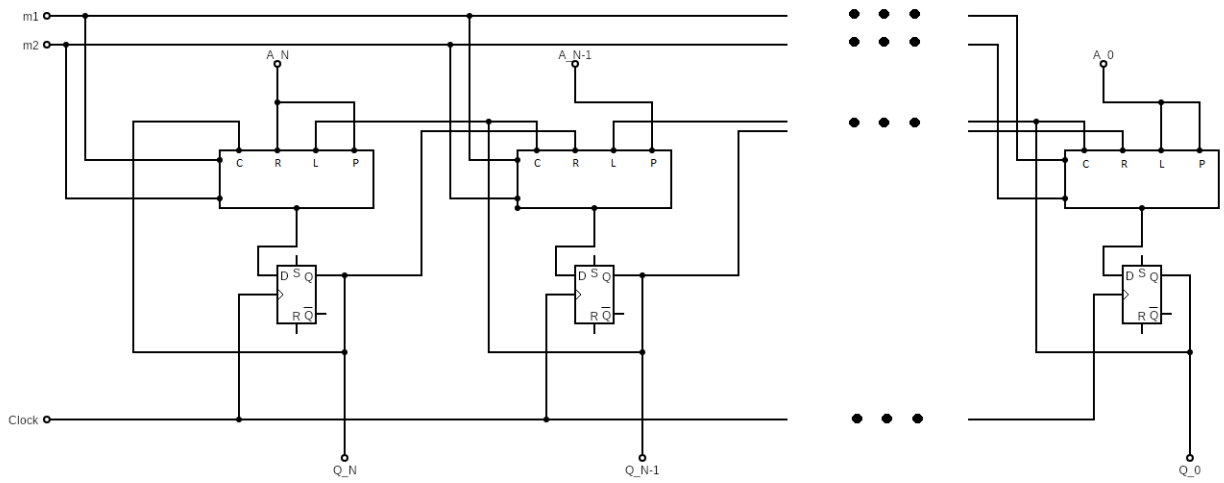


Figure 26: Parallel in, parallel out, n-bit, bidirectional shift register.

Just to explain how the circuit has been devised, the two modes run parallel over the multiplexers to enter from the sides. The parallel load stands above the the multiplexer, and on either end this acts as the parallel load input, and the right/left input respectfully (whilst for all multiplexers in between them, they only act as the parallel load input). Therefore the shift aspect, N-bit register aspect, the bidirectional aspect, and the parallel in aspect are all accounted for. The only thing left, is the parallel out, seen by the Q leaving the flip flop. Thus, concluding question 2.

3 Question 3.

Assume a function $F = A.B + \bar{A}.B.\bar{C}.D + \bar{A}.B.C.D + A.\bar{B}.\bar{C}.\bar{D}$**.**

3.1 a.

Reduce F to its simplest form using a Karnaugh map. You should show all working done to achieve your simplification.

To solve using a Karnaugh map, it was a simple process of laying out the table, and then place values where each expression holds.

CD \ AB	00	01	11	10
00				
01				
11				
10				

Case 1: $A.B$ is always true for the column of 11
Case 2: $\bar{A}.B.\bar{C}.D$ is only true for the coordinate (01,01)
Case 3: $\bar{A}.B.C.D$ is only true for the coordinate (01,11)
Case 4: $A.\bar{B}.\bar{C}.\bar{D}$ is only true for the coordinate (10,00)
This ends up making the final Karnaugh map look like this:

CD \ AB	00	01	11	10
00	0	0	1	1
01	0	1	1	0
11	0	1	1	0
10	0	0	1	0

Then by grouping all of the ones, we get these groupings:

CD \ AB	00	01	11	10
00	0	0	1	1
01	0	1	1	0
11	0	1	1	0
10	0	0	1	0

Now, when looking at the groupings of the Karnaugh map, we can look at which variables change, and which remain the same. For example, for the green grouping in the top right corner, we can see that both C and D do not change as it only remains over one row. Then we see that A is equal to 1 for all of the grouping, whilst B changes between 1 and 0. Therefore the final expression ends up being $(A \wedge \bar{C} \wedge \bar{D})$. Here C and D are both negated as the grouping holds for when they are equal to 0. By performing a similar process for the remaining groupings, and combining the expressions with ORs, we get the final expression of:
$F = (B \wedge D) \vee (A \wedge B) \vee (A \wedge \bar{C} \wedge \bar{D})$

The colors were used to distinguish between each grouping.

3.2 b.

Reduce F to its simplest form using Boolean algebra. You should show all working done to achieve your simplification.

Note: I used the $\vee$ and $\wedge$ symbols so I do not mess up simplification.
Note: Colored parts are what I am focusing on in each step.

$F = (A \wedge B) \vee (\bar{A} \wedge B \wedge \bar{C} \wedge D) \vee (\bar{A} \wedge B \wedge C \wedge D) \vee (A \wedge \bar{B} \wedge \bar{C} \wedge \bar{D})$
Commutative law:
$= (\bar{A} \wedge B \wedge \bar{C} \wedge D) \vee (\bar{A} \wedge B \wedge C \wedge D) \vee (A \wedge \bar{B} \wedge \bar{C} \wedge \bar{D}) \vee (A \wedge B)$
Distributive law:
$= \bar{A} \wedge ((B \wedge \bar{C} \wedge D) \vee (B \wedge C \wedge D)) \vee (A \wedge \bar{B} \wedge \bar{C} \wedge \bar{D}) \vee (A \wedge B)$
Commutative law:
$= \bar{A} \wedge ((B \wedge D) \wedge \bar{C} \vee (B \wedge D) \wedge C) \vee (A \wedge \bar{B} \wedge \bar{C} \wedge \bar{D}) \vee (A \wedge B)$
Distributive law:
$= \bar{A} \wedge ((B \wedge D) \wedge (\bar{C} \vee C)) \vee (A \wedge \bar{B} \wedge \bar{C} \wedge \bar{D}) \vee (A \wedge B)$
Complement law:
$= \bar{A} \wedge ((B \wedge D) \wedge 1) \vee (A \wedge \bar{B} \wedge \bar{C} \wedge \bar{D}) \vee (A \wedge B)$
Identity law:
$= (\bar{A} \wedge B \wedge D) \vee (A \wedge \bar{B} \wedge \bar{C} \wedge \bar{D}) \vee (A \wedge B)$
Commutative law:
$= (\bar{A} \wedge B \wedge D) \vee (A \wedge B) \vee (A \wedge \bar{B} \wedge \bar{C} \wedge \bar{D})$
Distributive law:

$= B \wedge ((\bar{A} \wedge D) \vee A) \vee (A \wedge \bar{B} \wedge \bar{C} \wedge \bar{D})$
Distributive law:
$= B \wedge (((\bar{A} \vee A) \wedge (D \vee A))) \vee (A \wedge \bar{B} \wedge \bar{C} \wedge \bar{D})$
Complement law:
$= B \wedge ((1 \wedge (D \vee A))) \vee (A \wedge \bar{B} \wedge \bar{C} \wedge \bar{D})$
Identity law:
$= B \wedge (D \vee A) \vee (A \wedge \bar{B} \wedge \bar{C} \wedge \bar{D})$
Distributive law:
$= (B \wedge D) \vee (A \wedge B) \vee (A \wedge \bar{B} \wedge \bar{C} \wedge \bar{D})$
Commutative law:
$= (A \wedge B) \vee (A \wedge \bar{B} \wedge \bar{C} \wedge \bar{D}) \vee (B \wedge D)$
Distributive law:
$= A \wedge (B \vee (\bar{B} \wedge \bar{C} \wedge \bar{D})) \vee (B \wedge D)$
Distributive law:
$= A \wedge ((B \vee \bar{B}) \wedge (B \vee (\bar{C} \wedge \bar{D}))) \vee (B \wedge D)$
Complement law:
$= A \wedge (1 \wedge (B \vee (\bar{C} \wedge \bar{D}))) \vee (B \wedge D)$
Identity law:
$= A \wedge (B \vee (\bar{C} \wedge \bar{D})) \vee (B \wedge D)$
Distributive law:
$= (A \wedge B) \vee (A \wedge \bar{C} \wedge \bar{D}) \vee (B \wedge D)$
Commutative law:
$= (B \wedge D) \vee (A \wedge B) \vee (A \wedge \bar{C} \wedge \bar{D})$

Which is the simplest form that the Karnaugh map found.

3.3 c.

Design a logic circuit that implements your simplification using only 2-input NAND gates.

3.3.1 Circuit Identities

For this question, it was a similar process to how question 1's logic gate question, by using the identities provided by NAND gates to rewrite the given expression. However, what is different is that the expression con-

sists AND, OR and NOT gates, requiring more identities.

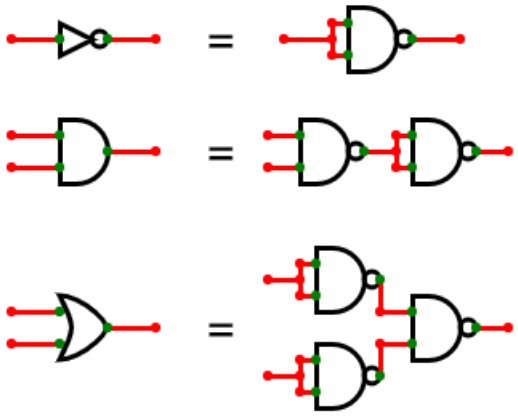


Figure 27: How to convert the basic gates into NAND gates.

3.3.2 Basic Gates Logic Circuit

Once again, I started with the usual process, mapping out the logical expression until it matched the simplified Boolean expression found before:

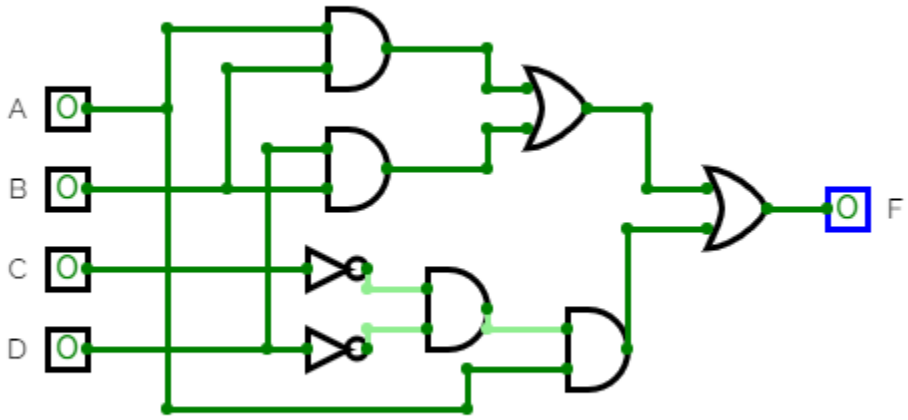


Figure 28: Logic circuit using the basic gates.

However, before I was about to go and mindlessly replace all of the gates with the NAND counterparts, I saw one simplification I could already

make to reduce the number of NAND gates (which you always want to strive for - see Q1).

3.3.3 NAND Gate Logic Circuit

The simplification concerns $((B \wedge D) \vee (A \wedge B))$ as I noticed something with NAND logic (Note: NAND will be represented using a Sheffer Stroke: $\uparrow$).
$x \uparrow y \equiv \neg(x \wedge y) \equiv \neg x \vee \neg y$
This on its own is meaningless, until we say rewrite our current expression:
$\overline{((B \wedge D) \vee (A \wedge B))}$
$\equiv \overline{(\overline{(B \wedge D)} \wedge \overline{(A \wedge B)})}$
Which is true because of Demorgan's law.
Upon inspection of the insides of the expression, we can see $\overline{(B \wedge D)}$ and $\overline{(A \wedge B)}$ are identical to the definition of NAND, so we can rewrite this too:
$\equiv \overline{((B \uparrow D) \wedge (A \uparrow B))}$
And when we look at this, we can see there is actually another dormant NAND. This again can be simplified too:
$\equiv (B \uparrow D) \uparrow (A \uparrow B)$
Boiling half of the expression down to 3 NAND gates!

Unfortunately, that is where my intuition ended, and I proceeded to fill out the expression as normal. Which ended up in producing this circuit:

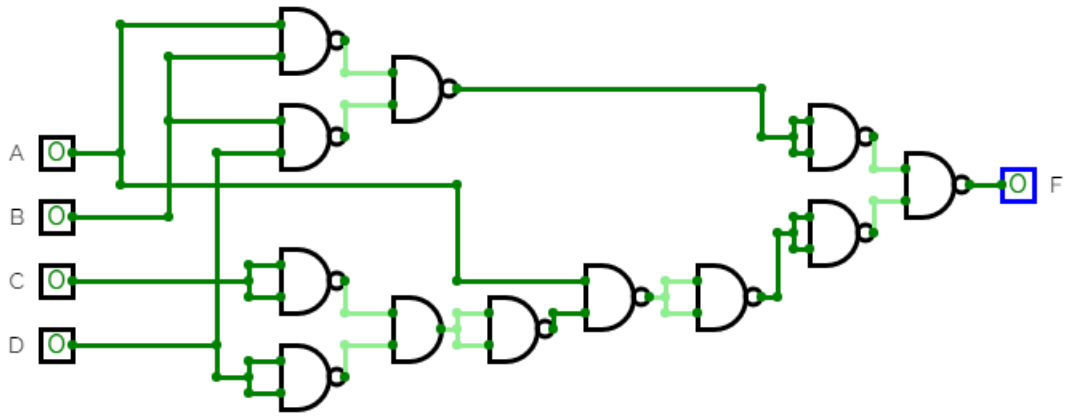


Figure 29: Logic circuit using only NAND gates.

Whilst I originally thought this was all I could do at first, writing out the expression in the form of gates did enlighten me to one area that wasn't optimized. On the final branch to F, there are two NAND gates (acting as NOT gates) which implies a double negation. By identifying this I was able to remove another two NAND gates, optimizing the circuitry even further:

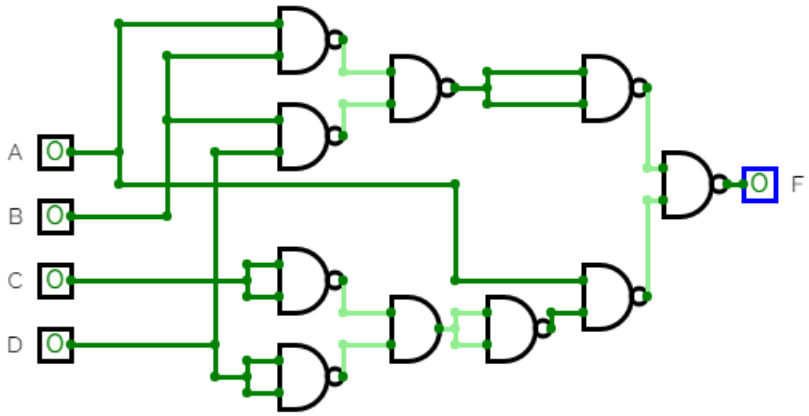
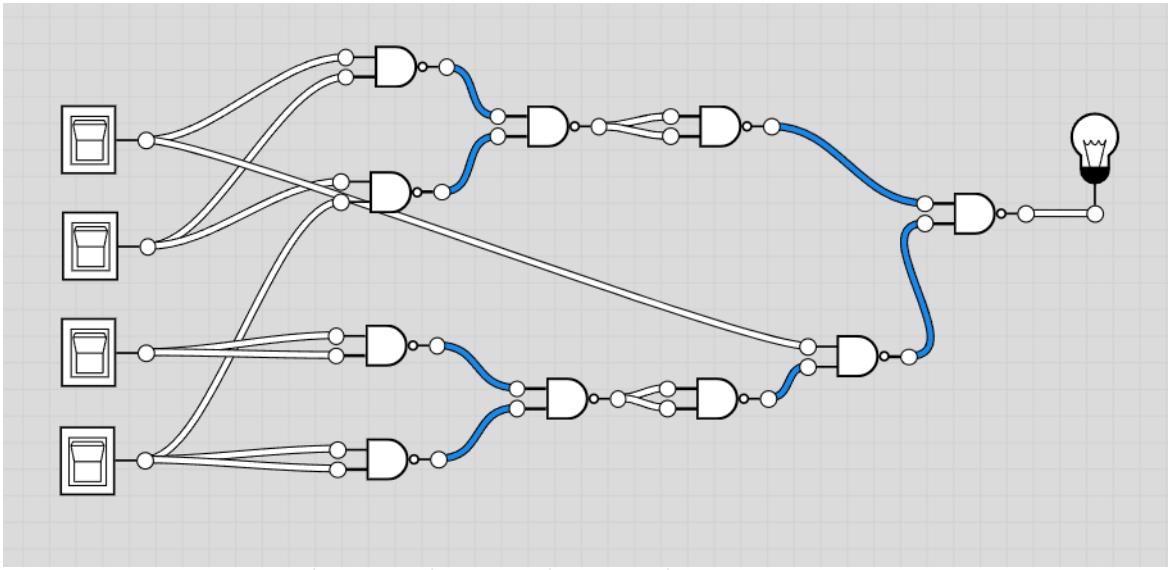


Figure 30: Refined logic circuit using only NAND gates.

Hence, I managed to finish designing the circuit: by not only applying Boolean algebra laws, but also seeing the expression simplify with the gates in front of me. It ended up at 10 NAND gates and hopefully that is as optimally designed as possible, yet there is a chance for other modifications to reduce the number of gates even more.

3.3.4 Testing

The final thing to do to confirm this is correct, is to test the circuit. This was done using logicly as they have an inbuilt feature to produce a truth table. The truth table that it produces from the circuit diagram below is as follows:



A	B	C	D	F
FALSE	FALSE	FALSE	FALSE	FALSE
FALSE	FALSE	FALSE	TRUE	FALSE
FALSE	FALSE	TRUE	FALSE	FALSE
FALSE	FALSE	TRUE	TRUE	FALSE
FALSE	TRUE	FALSE	FALSE	FALSE
FALSE	TRUE	FALSE	TRUE	TRUE
FALSE	TRUE	TRUE	FALSE	FALSE
FALSE	TRUE	TRUE	TRUE	TRUE
TRUE	FALSE	FALSE	FALSE	TRUE
TRUE	FALSE	FALSE	TRUE	FALSE
TRUE	FALSE	TRUE	FALSE	FALSE
TRUE	FALSE	TRUE	TRUE	FALSE
TRUE	TRUE	FALSE	FALSE	TRUE
TRUE	TRUE	FALSE	TRUE	TRUE
TRUE	TRUE	TRUE	FALSE	TRUE
TRUE	TRUE	TRUE	TRUE	TRUE

Figure 31: The testing process for the final circuit design.

And fortunately, this truth table holds, as if we were to input any value into out previous expression, we get a matching value from the table to be produced.

4 Question 4.

The algorithm below computes the greatest common divisor (GCD) of two positive integers. Input: Two positive integers, a and b Output: GCD(a,b)

```
while (b! = 0) do
 if (a > b) then
        a = a-b
    else
        b = b-a
    end
end
return a
```

4.1 a.

Write a C program that implements the GCD algorithm. Your program should provide feedback on what calculations are being performed and show a correct result. You should ensure your program is well documented and any error cases are handled appropriately. Clearly state any assumptions.
Note: All code segments in this documentation do not contain comments as to not fill the formatting. All of the comments for the final programs are in the C files attached with this PDF.

4.1.1 Introduction

When first reading the problem, I was fairly content and happy in understanding how to implement it, however the more maths oriented side of me had its interest piqued. The main area that got my interest was surrounding the two positive integers, a and b. Was GCD limited to only positive integers? And secondly, I was wondering if this was the most efficient method to calculate the GCD of a and b, or is there some way that perhaps calculated it better?

4.1.2 Initial Research and Design

But before I spent my time experimenting with the interesting mathematics surrounding the program, I thought it would be more beneficial to implement it as intended, and fortunately it was fairly easy to do. Despite being given the program, after all of my research and past experiences with the program, I saw there was an alternate method using

mod. Both of which are the same - Euclid's method - but this method theoretically requires less steps, requires no if statement, and most importantly easier to write up. (Minus signs were awkward to write in the code format for latex, hence minimizing all use of minus signs were preferred).

The plan then made was to implement the GCD function, and then proceed to filter out any invalid inputs and have error handling in separate functions/main. Firstly was the step of deciding how to implement Euclid's algorithm. There was a way using a while loop, and a way doing it recursively:

```
int GCD_While(int num1, int num2){
  while (num2 != 0){
    int temp = num2;
    num2 = num1 % num2;
    num1 = temp;
  }
  return num1;
}
```

```
int GCD_Recursive(int num1, int num2){
  if (num2 == 0){
    return num1;
  }
  else {
    return GCD(num2, num1 % num2);
  }
}
```

Both of these functions perform the same logic, only the difference is how they keep repeating. Whilst for the most part indistinguishable, there is one with an overwhelming advantage. That is the the while loop method. This is because the way recursion works is that it continually adds to the stack the new data values before it can pop them all off at the end, whereas the while loop just updates existing memory values. Therefore this method is more memory efficient, and hence that is the way the function is written.

4.1.3 Validation

When being given the algorithm, it makes it difficult to explain the thinking process behind each step, however what does need that is input validation. Currently, we only wish to accept positive integers, meaning we want to reject any input that is not an integer, or any input that exceeds the maximum size of the input. This first part can be solved with the

help of inbuilt functions of C, as this line of code can be used to root out all non-integers:

```
if (scanf("%d%c", &num1, &enter) != 2 || enter != '\n')
```

If this statement returns true, then the input gathered by scanf is not an integer. Here is how it is broken down:
- 'scanf("%d%c", num1, enter)': This returns the number of matches found with the inputs given. Here it is looking for an integer and a character.
- '(scanf("%d%c", num1, enter) != 2': This implies that both the number entered, and the character has to actually be what is intended, otherwise it is an invalid input.
- 'enter != "' However, it has to be specified that the character can only be a newline character. This is indicated by the user pressing enter (hence the variable name).
- By combining these elements, we get a statement that will reject all non integers.

4.1.4 First Design

At first I had tried to only use a single check condition, comparing scanf("%d") != 1 however this only checked that the first few digits were integers, as if you entered 11231A, then it woul still go ahead and treat it as if it were correct. With the new modifications, it works as intended! Almost.

```
#include <stdio.h>

int GCD(int num1, int num2){
  while (num2 != 0){
    int temp = num2;
    num2 = num1 % num2;
    num1 = temp;
  }
  return num1;
}

int main(){
  int num1;
  int num2;
  char enter;
  char enter2;

  printf("Enter num1: \n");
  if (scanf("%d%c", &num1, &enter) != 2 || enter != '\n')
  {
    printf("Invalid input \n");
  }
  else {
    printf("Enter num2: \n");
```

```
25      if (scanf("%d%c", &num2, &enter2) != 2 || enter2 != '\n')
26      {
27        printf("Invalid input \n");
28      }
29      else {
30        printf("GCD of %d and %d is %d \n", num1, num2, GCD(num1, num2));
31      }
32    }
33
34  }
```

Above is the complete program to go and calculate the GCD of two positive integers, and while the main function could be written neater using return 0 in the if statements to reduce nesting, the functionality of it works, and it gives expected outcomes by testing values against an online calculator. However, it isn't yet complete.

4.1.5 Size Limits

The first issue we have to address is regarding maximum integer sizes. In C, an overflow will occur if the value exceeds 2147483647 (or the negative version of this). For example what we expect to be 2147483648, actually becomes -2147483648; or 21474836444343 becomes -35657. Therefore, we need to remove the possibility of inputting these values into the function. The problem is... its tricky. There are ways I can do it, with issues with all of them.
I can take the input as a double, and then I can test if it is greater than the maximum value; if it is then I reject it, if it isn't then I make the double an integer again.
This method is an appealing one, however it completely makes my other workings redundant as the awkwardness of C makes it difficult to reuse scanf to retest certain things. Therefore I would have to redo all of my other validation tests, which is problematic as the only other method I can see is to treat it like a string and check each and every character. That wasn't going to work.

All ways of making it so my data was limited in size, meant it would not be able to remove error cases of non inputs. Whilst I am sure that a solution exists to this problem, it is one I am unable to solve. However, I am going to prioritize the implementation of removing strings and other illogical inputs, rather than removing overflow errors. The reason for this is partially because the code for the removal of values exceeding the max

integer length is frustrating, requiring the initial input of a double, back to an integer (so there is even more risk for error), however the main reason, is because it doesn't hurt the output of the program as much. At the end of my program is a print statement that says the GCD of num1 and num2 is x, and if num1 has changed then so has that output, and technically that will be correct, even though it will not be what the user wanted. Additionally, it is more common for people to make typing errors, than it will be for people to exceed the integer limit. Regardless, this isn't perfect, and definitely can be improved if I find another way to remove erroneous cases from integers.

4.1.6 GCD and 0

Another possible error case to look into is what happens when you enter 0 as an input. If we enter 0 into the program we see that $\gcd(0, a) = \gcd(a, 0) = a$ which may be nice for our program, but is that actually correct? 0 is always a weird number, but there is an argument to make that what my program outputs is correct. The GCD of a pair of numbers, is the greatest number that divides both of them (giving an integer output). If we take the case of 0 and a, if we say that 0 is an integer, then 0 divided by anything is 0, and hence will always give an integer regardless of GCD. Therefore we only need the greatest divider of a, which will always be a. Therefore, the solution makes sense! Despite this, there is an exception. The $\gcd(0, 0)$ is a big mystery. If we applied the logic of $\gcd(0, a), a = 0$ we get that $\gcd(0, 0) = 0$ which is problematic as that brings in the question of $\frac{0}{0}$. However, if we take a step back and look at the definition of GCD, we can see that 0 makes the least amount of sense. As previously stated, $\frac{0}{a} = 0$, so regardless of how big a is, an integer will always be produced, so we should take the greatest divisor of the other number, but if that is also the case for the other number, that means we want the highest a possible. Since a can continually keep growing we can say that the $\gcd(0, 0) = \infty$ Therefore, we want to add a condition on the program saying if both num1 and num2 equal 0, it is to print "infinity". Is what I originally thought. Thought after doing some research, it is suggested that in fact $\gcd(0, 0) = 0$, the logic of which being more of a convention, rather than a definition. This is so that the function of GCD can maintain certain properties, such as

$m \gcd(a, b) = \gcd(am, bm)$. Therefore, I do not need to amend the code.

4.1.7 Extending the Domain

The potential errors do not stop there though, as my current code accepts all integers, which happen to include the negative ones. Fortunately, there is a simple fix which allows for this to not be the case, and that is to expand the domain of the problem.

Whilst I could include an if statement that disallows all numbers less than 0, what is more interesting is the GCD of negative numbers. In my research I found that this identity holds:
$\gcd(x, y) = \gcd(-x, y) = \gcd(x, -y) = \gcd(-x, -y)$
That is to say,
$\gcd(x, y) = \gcd(|x|, |y|)$
However, considering the sources of research varied between Wikipedia and stack overflow pages, this formula shouldn't be just blindly accepted. That is why it is fortunate that there is a proof to nicely justify it:

Note: $x \backslash y$ states that y is divisible by x.

Suppose $u \backslash a$.
$\implies \exists q : a = qu$
$\therefore -u \backslash a \implies a = (-q)(-u)$ and $u \backslash -a \implies -a = (-q)u$
What this tells us is that every divisor of a is a divisor of both positive and negative a, so every divisor of a is a divisor of $|a|$.
Hence it follows that any common divisor between x and y, is a common divisor of $|x|$ and $|y|$. As GCD is the greatest common divisor, and all common divisors are shown to hold for positive and negative integers, we can conclude $\gcd(x, y) = \gcd(|x|, |y|)$
With this conclusion, we can now make it so in the program, that it will take the absolute of the values when the function is called, still availing the same result. The only change needing to be made is the final line in which GCD is called. Here, I pass the absolute value of num1 and num2.

```
1 printf("GCD of %d and %d is %d \n", num1, num2, GCD(abs(num1), abs(num2)));
```

Where luckily, abs is a built in function to C in the ¡stdlib.h¿ library, taking the absolute value of any input. Therefore with this, the base project of the GCD program is complete, with the only issue being it cannot handle numbers outside the range of -2147483648 to 2147483647.

```c
#include <stdio.h>
#include <stdlib.h>

int GCD(int num1, int num2){
  while (num2 != 0){
    int temp = num2;
    num2 = num1 % num2;
    num1 = temp;
  }
  return num1;
}

int main(){
  int num1;
  int num2;
  char enter;
  char enter2;

  printf("Enter num1: \n");
  if (scanf("%d%c", &num1, &enter) != 2 || enter != '\n')
  {
    printf("Invalid input \n");
  }
  else {
    printf("Enter num2: \n");
    if (scanf("%d%c", &num2, &enter2) != 2 || enter2 != '\n')
    {
      printf("Invalid input \n");
    }
    else {
      printf("GCD of %d and %d is %d \n", num1, num2, GCD(abs(num1), abs(num2)));
    }
  }

}
```

4.1.8 Justifications

What there is to note, is that the way that the validation works is that it closes the program upon an invalid input. This is because trying to use scanf and while loops in C aren't so nice. There was the constant error of the scanf function not waiting for user input, just flooding the console with repeated text. Therefore, I decided it wasn't worth spending too much time on, as there were more important areas to focus on.

4.1.9 Further Research

Whilst the program is complete, it still doesn't alleviate my curiosity of non-integer GCD. Getting it to work for negative numbers was a nice extension to working values, however I am to make the far more ambitious claim that you can take the GCD of any two numbers in the set of

rational numbers.

When we think of the GCD of two numbers, we don't commonly associate a formula with it. That is because the one for it, isn't particularly useful. While Euclid's algorithm (the program referenced in the question) is an efficient way to calculate the GCD, it isn't an explicit formula. An actual formula to calculate GCD actually comes from prime numbers. Before showing the formula it requires some base understanding.

4.1.10 Fundamental Theorem of Arithmetic

Firstly, the fundamental theorem of arithmetic explains how any number can be expressed as a product of primes. This is known as prime factorisation, for example:
$18 = 2 \cdot 9 = 2 \cdot 3 \cdot 3 = 2 \cdot 3^2$
Prime factorisation is the process of boiling a number into its base components of primes. This can be more formally represented like this:

$$n = 2^{n_2} \cdot 3^{n_3} \cdot 5^{n_5} \cdot 7^{n_7} \cdot ... = \prod_{p\,primes}^{\infty} p^{n_p}$$

Note: All values that aren't used, are just 0; for example:
$18 = 2^1 \cdot 3^2 \cdot 5^0 \cdot 7^0 \cdot ...$
By using this fact, a proof can be made to show that:

$$\gcd(a, b) = \prod_{p}^{\infty} p^{\min(a_p, b_p)}$$

This formula isn't going to be proven here, as it is quite long, but it is referenced in the bibliography, with the name of the paper being 'Greatest common divisor as a product of primes'.

4.1.11 Example

Before proceeding, an example with the formula may help. Suppose we find $\gcd(16, 20)$. Using Euclid's algorithm, we get:
$a = 16, b = 20$
$a < b : b = 20 - 16 = 4$
$a = 16, b = 4$
$a > b : a = 16 - 4 = 12$

$a = 12, b = 4$
$a > b : a = 12 - 4 = 8$
$a = 8, b = 4$
$a > b : a = 8 - 4 = 4$
$a = 4, b = 4$
$a < b : b = 4 - 4 = 0$
$b = 0 \therefore \gcd(16, 20) = a = 4$

Whereas using the formula, first we write the inputs as a product of prime factors:
$16 = 2^4$
$20 = 2^2 \cdot 5^1$
$\gcd(16, 20) = 2^{\min(4,2)} \cdot 5^{\min(0,1)} = 2^2 \cdot 5^0 = 2^2 = 4$

4.1.12 Extending the Domain to the Rationals

With this formula, we can use this to extend the definition of GCD. When we look at a rational number, we can express it in the form $\frac{m}{n} : n, m \in \mathbb{Z}$. Then if we use the other fact that all integers can be expressed as a product of primes, we can make a new definition:
$a = \frac{m}{n} : n, m \in \mathbb{Z}$
$= \frac{2^{m_2} \cdot 3^{m_3} \cdot 5^{m_5} \cdot ...}{2^{n_2} \cdot 3^{n_3} \cdot 5^{n_5} \cdot ...}$
$= \frac{2^{m_2}}{2^{n_2}} \cdot \frac{3^{m_3}}{3^{n_3}} \cdot \frac{5^{m_5}}{5^{n_5}} \cdot ...$
$= \overset{\infty}{2}^{m_2 - n_2} \cdot 3^{m_3 - n_3} \cdot ...$
$= \prod_p p^{m_p - n_p}$
This implies that all rational numbers can be expressed as a multiplication of primes. The only difference is that the exponents may not be positive integers, yet it holds nevertheless. For example:
$\frac{2}{9} = 2^1 \cdot 3^{-1} \cdot 3^{-1} = 2^1 \cdot 3^{-2}$
What this all allows us to conclude is that:
$\forall x \in \mathbb{Q}, x_p = m_p - n_p : x = \frac{m}{n}, m, n \in \mathbb{Z}$
And so, the GCD of any two rational numbers can be found!

For an example we will find: $\gcd\left(\frac{8}{9}, 12\right)$
$\frac{8}{9} = 2^3 \cdot 3^{-2}$
$12 = 2^2 \cdot 3^1$
$\gcd\left(\frac{8}{9}, 12\right) = 2^{\min(3,2)} \cdot 3^{\min(-2,1)} = 2^2 \cdot 3^{-2} = \frac{4}{9}$
The idea of doing the greatest common divisor between non-integers seems plain incorrect, something about it not making much sense, however by exploring the behaviour of a couple answers I can see some patterns. The answer given, when divided through a and b, both give integer outputs; and additionally the value given is the greatest common divisor, that gives an integer output. Obviously this is not yet rigorously proven, and nor is it even seeming to be a pattern case that can be easily tested since it takes so much more work to find out. That is why, it needs to be programmed.

4.1.13 Algorithmic Approaches

To program the GCD of rational numbers though is a far more complex process than I would have thought, especially in something like C. The program would take 4 inputs: $n_1 = \frac{a}{b}, n_2 = \frac{c}{d}$ where a,b,c,d are the inputs. Then a 2D array would need to be used, in which it would express all non zero exponents, for example if $a = 18$, then $pfactor(a) = ((2,1),(3,2))$, as to represent $a = 2^1 \cdot 3^2$. Then this array and the array of b would be combined where if there are matching prime values, it would subtract the exponent. Eventually two 2D arrays of prime numbers and exponents would be produced for each fraction, and finally this would need to be traversed using the min function on the exponents.

Quite frankly, in C that is really hard to do. Whilst not impossible, it is incredibly difficult to accomplish effectively without a dynamic data structure, one which C cannot offer - at least easily. That is why it is unfortunate to say that the program was not produced, as not only would

it be highly inefficient for prime factorisation having few short cuts, but also extremely difficult to maneuver using C's memory system. But, at the very least, the mathematics behind it was very interesting.

4.1.14 Conclusion

To conclude, the C program that was originally produced following the initial guidance and spec is the most useful to the user. Not only is that what was asked for, wanting two positive integers, but it is also the most commonly used. Rarely, have I seen the need for the GCD of two fractional numbers. Additionally, the way that it would need to be calculated would be quite difficult for the machine, having to prime factorize numbers. Really, it is far less efficient to use, and far less useful in near all situations.

4.2 b.

Write a C program that calculates the sum of two 8-bit binary strings represented in sign-magnitude, printing the result of the calculation in two's complement. You should assume that both inputs are null-terminated. Your program should provide feedback on what calculations are being performed and show a correct result. You should ensure your program is well documented and any error cases are handled appropriately. Clearly state any assumptions.

4.2.1 Decomposition

Upon first glance at this problem, I was feeling far more confident. This problem was one that required more computational thought, in figuring

out an optimal way to produce the output intended. The first thing considered was two different methods I could pursue:
1. I could convert the two sign and magnitude inputs into denary, and then add them using C's addition, and then convert back into two's complement's form, and output that to the user.
2. Or, I could convert the two sign and magnitude inputs into two's complement form, and then perform binary addition on the two binary strings, and output that to the user.

4.2.2 Method Comparison

Method 1 and 2 were equally valid, but there were certain advantages and disadvantages to each.

Method 1 proposed a solution that made things far easier for me, and for the user. This is because converting from sign and magnitude to denary is a very easy formula to follow. Additionally, when adding the two values together there is no risk of failure from overflow, and finally when outputting, I do not need to exclude potential values as I can output in as many bits as desired.

Method 2 meanwhile is conceptually the better program as it should require far less operations from the computer. When converting back and forth between binary and denary as done in method 1, it is done with the caveat that I will need to use multiple exponents, resulting in a lot of repeated multiplication. Whereas method 2 will simply need to input 3 values into a function, and it will be done. The implementation of a full adder would be used here, and the conversion of sign and magnitude to two's complement would be easy, by toggling the bits and adding one (to those with a 1 as the MSB).

Both have their own distinct advantages, but in terms of which would be easier to program, it would definitely have to be method 1. Method 2 requires a much greater understanding of C memory architecture and how it handles bit-wise operations; information I do not have. Furthermore, the ability to be able to output more values without overflow error is far more useful to the user. This is the case because when considering the range of sign and magnitude, and two's complement, we see that by adding 01111111 + 01111111, this will need to go into 9 bits, as it is the equivalent of 127+127=254, so we will need the 8th bit free to make this number in two's complement. This choice is lost with method 2, and so I am to go down the route of method 1.

4.2.3 Further Decomposition

Designing the program, I need to break it down into each module:
- toDenary function: this will convert the 8 bit sign and magnitude inputs into denary.
- toBinary function: this will convert the denary value into a 9 bit binary value that follows two's complement.
- inputValidation function: this will check that the string the user inputted is actually valid, and let the program know it can proceed.

4.2.4 toDenary Function

The first step is the toDenary function. This will need to take a string of length 8 (ignoring the terminating value), and go through each character in the array, convert it to an integer, in which it can be converted. As you can tell, I quite like maths, so I wrote it in this more readable form:
$A = [A_0, A_1, A_2, ..., A_6, A_7]$

$$\text{denary} = (-1)^{A_0} \cdot (\sum_{i=1}^{7} A_i \cdot 2^{7-i})$$

If this was to be implemented in code, this would require an exponential function. There was one I could import using the C library, however that makes compilation slightly more awkward for the marker so it was avoided. Additionally, the power function that used was far more than this program needed. As I would only be dealing in integer exponentiation, not using doubles like that library one. So first a very simple exponential function was implemented like so:

```
int power(int x, int y)
{
  int total=1;
  int i;
  for (i = 0; i < y; ++i)
  {
    total = total * x;
  }
  return total;
}
```

This function quite simply, repeatedly multiplies a number y amount of times. If to be expressed in mathematical form:

$power(x, y) = x^y$

The function may not have worked for non integer values, and negative exponents, but for this program, it didn't need to.

With the new power function, the toDenary function could be written, in which it very simply implemented the mathematical formula as previously stated:

```
int toDenary(char bin[])
{
  int i;
  int denary = 0;
  for (i = 1; i < 8; ++i)
  {
    int num = bin[i] - '0';
    denary = denary + num * power(2, 7-i);
  }
  int sign = bin[0] - '0';
  denary = denary * power(-1, sign);
```

```
12   return denary;
13 }
```

The only notable points to discuss here is how the characters were converted to integers. This was done by writing: bin[i] - '0', which is a simple conversion of a character to an integer.

4.2.5 toBinary Function

The next large module to complete was the toBinary function. This would be slightly more complex to implement. The pseudocode logic for the program would follow as so:

```
toBinary(denary):
    for i from 7 to 0:
        if (denary - 2^i) < 0 then
            binary[7-i] = 0
        else then
            binary[7-i] = 1
            denary = denary - 2^i
        end if
    next i
```

The method described above is a common way of figuring out binary from denary, with fairly easy to understand. If the program itself was this simple, it would have been so much easier. However, C has the reoccurring problem of difficult to understand memory.

There were two very C related problems I had to overcome. The two manifested themselves in the form of extra random characters at the end of my string when printed, much of the time the characters having no key representation. This was strange, and it was all happening because of how I was trying to return my string. To return a string from a function in C, the function has to be declared as a "const char *". From there within the function I set the string bin[8], and I returned bin:

```
const char* toBinary(int den)
{
  int i;
  char* bin[8];
  for (i = 7; i >= 0; --i)
```

```
  {
    if ((den - power(2,i)) < 0)
    {
      bin[7-i] = '0';
    } else {
      bin[7-i] = '1';
      den = den - power(2, i);
    }
  }
  return bin;
}
```

This code though continually returned error after error. I tried returning bin, or *bin, much to my confusion and lack of understanding of C pointers. However, eventually I stepped back and I reconsidered what I was doing. What was really going on was that I was returning the address of a local variable. The variable of bin was local to the function, and so returning that temporarily existing address caused all sorts of problems, as the variable no longer existed anymore. Also, I entirely forgot about adding the null terminating character. So I increased the size of the array, and set the null terminating character at the end, and then I proceeded to use a cheat, of sorts. Including the library ¡stdlib.h¿, I could use a command called malloc - or memory allocate - which, to grossly oversimplify, makes the variable part of the heap. The heap, to my understanding, is in a way like global memory, and so whilst it is ineffective to use it this way, I found this to be the best way to deal with the errors it was causing. The parameters of malloc takes the size of the data type, and then the size of the memory block, in this case being the string array, of 9 characters. This resulted in the updated code looking like:

```
const char* toBinary(int den)
{
  int i;
  char* bin = malloc(sizeof(char) * 9);
  for (i = 7; i >= 0; --i)
  {
    if ((den - power(2,i)) < 0)
    {
      bin[7-i] = '0';
    } else {
      bin[7-i] = '1';
```

```
      den = den - power(2, i);
    }
  }
  bin[8] = '\0';
  return bin;
}
```

4.2.6 validation Function

With that complete, we were down to the final module of the input validation. As it was working with strings instead of integers, I felt far more confident in writing it, and it shows with the very focused and condensed function. The validation would take in a a string, and then it would perform 3 checks:

1. Check length
2. Check all values are either 0 or 1
3. Check for all white spaces

The first two points are mostly self explanatory, as they are what are required to be binary. The length could be checked using the library ¡string.h¿ with the command strlen(). However the third point is slightly more obscure. Without checking for white spaces, if you put a space then it would only factor the first word, not the one after the space. This wasn't the end of the world, but it could have been written much better. So, by checking if there is a white space instead of the terminating character, this will find out if the word it has seen is the only word. Once it knows it is, the function will simply return 1 to say it is valid, or 0 to say it is invalid.

```
int validation(char bin[])
{
  if (strlen(bin) == 8)
  {
    int i;
    for (i = 0; i < 8; ++i)
    {
      if ((bin[i] != '1' && bin[i] != '0') || bin[i] == ' ')
      {
        return 0;
      }
    }
```

```
13      return 1;
14    } else {
15      return 0;
16    }
17  }
```

4.2.7 The Sum of its Parts

Now all that was left to do, is combine all of these functions and give the final solution. This wasn't as simple though as shoving all of the inputs in and out of the functions, as with any good problem, maths was in the way. The way that the convert to binary function worked is that it converted assuming it is positive, however since it is two's complement we need to account for when it is negative. Originally I was going to convert to two's complement, then flip the bits and then add 1 in binary, but by doing that I undermined the way I had been going about my program thus far, as I would have to introduce a binary adder function, which I was not going to do. So instead, I relied on the definition of two's complement.

$A = [A_0, A_1, A_2, ..., A_6, A_7]$

$$\text{denary} = -A_0 \cdot 2^8 + (\sum_{i=1}^{7} A_i \cdot 2^{7-i})$$

When looking at this formula we see that there is a constant amount that is subtracted from the denary value, as the MSB is always negative. So, I thought if the denary value is negative, then that means it will always have $A_0 = 1$, in which case we can say:

$$\text{denary} + 256 = (\sum_{i=1}^{7} A_i \cdot 2^{7-i})$$

Which is just a new denary value now, that we know how to convert the normal way. Therefore, we can make the conclusion that '1' + 'toBinary(denary+256)' (where the + is concatenating) will give two's complement. This means we can write the final statement of the program depending on if the sum is positive or negative. So, we can finally sum

together every part of the code to produce:

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

int power(int x, int y)
{
  int total=1;
  int i;
  for (i = 0; i < y; ++i)
  {
    total = total * x;
  }
  return total;
}

int toDenary(char bin[])
{
  int i;
  int denary = 0;
  for (i = 1; i < 8; ++i)
  {
    int num = bin[i] - '0';
    denary = denary + num * power(2, 7-i);
  }
  int sign = bin[0] - '0';
  denary = denary * power(-1, sign);
  return denary;
}

const char* toBinary(int den)
{
  int i;
  char* bin = malloc(sizeof(char) * 9);
  for (i = 7; i >= 0; --i)
  {
    if ((den - power(2,i)) < 0)
    {
      bin[7-i] = '0';
    } else {
      bin[7-i] = '1';
      den = den - power(2, i);
    }
  }
  bin[8] = '\0';
  return bin;
}


int validation(char bin[])
{
  if (strlen(bin) == 8)
  {
    int i;
    for (i = 0; i < 8; ++i)
    {
      if ((bin[i] != '1' && bin[i] != '0') || bin[i] == ' ')
```

```
      {
        return 0;
      }
    }
    return 1;
  } else {
    return 0;
  }
}


int main()
{
    char bin1[8];
    char bin2[8];

    printf("Enter first binary value: \n");
    scanf("%s", bin1);
    if (validation(bin1) == 0){
      return 0;
    }
    int val1 = toDenary(bin1);
    printf("= %d \n", val1);

    printf("Enter second binary value: \n");
    scanf("%s", bin2);
    if (validation(bin2) == 0){
      return 0;
    }
    int val2 = toDenary(bin2);
    printf("= %d \n", val2);

    int sum = val1 + val2;
    printf("%d + %d = %d \n", val1, val2, sum);
    if (sum < 0)
    {
      sum = sum + 256;
      printf("= 1%s \n", toBinary(sum));
    } else {
      printf("= 0%s \n", toBinary(sum));
    }

    return 0;
}
```

5 Bibliography

Reference list

All About Circuits (n.d.). Shift Registers: Parallel-in, Serial-out (PISO) Conversion — Shift Registers — Electronics Textbook. [online] www.allaboutcircuits.com. Available at: https://www.allaboutcircuits.com/textbook/digital/chpt-12/parallel-in-serial-out-shift-register/.

All About Circuits (n.d.). Shift Registers: Serial-in, Serial-out — Shift Registers — Electronics Textbook. [online] www.allaboutcircuits.com. Available at: https://www.allaboutcircuits.com/textbook/digital/chpt-12/serial-in-serial-out-shift-register/.

Circuit Diagram (2019). Circuit Diagram - A Circuit Diagram Maker. [online] Circuit-diagram.org. Available at: https://www.circuit-diagram.org/ Used for circuit diagrams.

Circuitverse (2019). CircuitVerse - Digital Circuit Simulator online. [online] Circuitverse.org. Available at: https://circuitverse.org/simulator Used for circuit diagrams.

corob-msft (2021). C and C++ Integer Limits. [online] docs.microsoft.com. Available at: https://docs.microsoft.com/en-us/cpp/c-language/cpp-integer-limits?view=msvc-170.

Electrical4U (2020a). Parallel in Parallel Out (PIPO) Shift Register — Electrical4U. [online] https://www.electrical4u.com/. Available at: https://www.electrical4u.com/parallel-in-parallel-out-pipo-shift-register/.

Electrical4U (2020b). Serial in Serial Out (SISO) Shift Register — Electrical4U. [online] https://www.electrical4u.com/. Available at: https://www.electrical4u.com/serial-in-serial-out-siso-shift-register/.

Engineering Funda (2021). Bidirectional Shift Register (Circuit, Block Diagram Working), Digital Electronics. [online] www.youtube.com. Available at: https://www.youtube.com/watch?v=K1lVwPRJDGQ.

GeeksforGeeks (2015). sizeof operator in C. [online] GeeksforGeeks. Available at: https://www.geeksforgeeks.org/sizeof-operator-c/.

Gracelin, P. (n.d.). 4 -bit bidirectional shift register with parallel load. [online] Available at: https://webadmin.sarahtuckercollege.edu.in/admin//Public/uploads/e-material/Physics%20Regular/Pushpalatha/4-%20Bit%20Bidirectional%20Shift%20Register.pdf.

javapoint (n.d.). Bidirectional Shift Register - javatpoint. [online] www.javatpoint.com. Available at: https://www.javatpoint.com/verilog-bidirectional-shift-register.

Leeke, M. (2021). Digital Logic. University of Warwick.

logic.ly. (n.d.). Logicly - A logic circuit simulator for Windows and macOS - logic gates, flip-flops, computer architecture, electronics, integrated circuits. [online] Available at: https://logic.ly/.

Mathematics Stack Exchange. (2011). abstract algebra - What is $\gcd(0, a)$, where a is a positive integer? [online] Available at: https://math.stackexchange.com/questions/27719/what-is-gcd0-a-where-a-is-a-positive-integer.

Mathematics Stack Exchange. (2013). elementary number theory - What is $\gcd(0, 0)$? [online] Available at: https://math.stackexchange.com/questions/495119/what-is-gcd0-0.

Newstead, C. (2014). Greatest common divisor as a product of primes. [online] Available at: https://www.math.cmu.edu/~cnewstea/teaching/old/teaching/21-127-S14/handouts/gcd-with-fta.pdf.

proofwiki.org. (n.d.). GCD for Negative Integers - ProofWiki. [online] Available at: https://proofwiki.org/wiki/GCD_for_Negative_Integers.

Thompson, B. (2021). C Bitwise Operators: AND, OR, XOR, Shift Complement (with Example). [online] www.guru99.com. Available at: https://www.guru99.com/c-bitwise-operators.html.

tutorialspoint (n.d.). C library function - malloc() - Tutorialspoint. [online] www.tutorialspoint.com. Available at: https://www.tutorialspoint.com/c_standard_library/c_function_malloc.htm.

Wikipedia. (2021). Euclidean algorithm. [online] Available at: https://en.wikipedia.org/wiki/Euclidean_algorithm#Procedure.

www.wolframalpha.com. (n.d.). gcd %280%2C0%29 - Wolfram—Alpha. [online] Available at: https://www.wolframalpha.com/input/?i=gcd+%280%2C0%29.